

CS7DS2 Assignment - Final

Name: Kehan Liu

ID:25334065

Q1

Q1.A

As required by the assignment, I implemented four optimization algorithms for benchmarks A, B, and C, and included a constant-step gradient descent (GD) as a comparison benchmark.

Below are the implementations of each algorithm:

1. Polyak Step Size

The Polyak step size is an adaptive global learning rate mechanism. It dynamically calculates the step size based on the difference between the current objective function value $f(x_k)$ and the known optimal value f^* to calculate the step size. The update formula is:

$$\alpha_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2 + \epsilon}$$

In the code, after calculating the step size, we apply it uniformly to all coordinate dimensions.

```
elif name == 'Polyak':
    f_val = loss_fn(x)
    f_star = params['f_star']
    eps = params['eps']
    alpha = (f_val - f_star) / (np.sum(g**2) + eps)
    step = alpha * g
    alpha_eff = np.array([alpha, alpha])
```

2. Adagrad

Adagrad provides adaptive step sizes for each coordinate. It reduces the learning rate for parameters that are updated frequently by accumulating the squares of historical gradients ($G += g**2$). To analyze its behavior in the report, my code fully records the effective step size α_k for each dimension.

```
elif name == 'Adagrad':
    alpha_0 = params['alpha_0']
    eps = params['eps']
    G += g**2
    alpha_k = alpha_0 / np.sqrt(G + eps)
    step = alpha_k * g
    alpha_eff = alpha_k.copy()
```

3. RMSprop

RMSprop is similar to Adagrad; RMSprop also uses coordinate-wise adaptive step sizes, but it employs an exponentially weighted moving average to accumulate the squares of historical gradients. This avoids the issue of Adagrad's step size decreasing too rapidly. The decay rate parameter β is introduced:

```

elif name == 'RMSprop':
    alpha_0 = params['alpha_0']
    beta = params['beta']
    eps = params['eps']
    G = beta * G + (1 - beta) * g**2
    alpha_k = alpha_0 / np.sqrt(G + eps)
    step = alpha_k * g
    alpha_eff = alpha_k.copy()

```

4. Polyak Momentum

The Polyak momentum method accumulates past gradient directions by introducing a momentum term z_k , thereby increasing the step size to accelerate along gentle slopes and decreasing the step size to suppress oscillations along steep slopes. Its update formula is:

$$z_{k+1} = \beta z_k + \alpha \nabla f(x_k)$$

$$x_{k+1} = x_k - z_{k+1}$$

```

elif name == 'Momentum':
    alpha = params['alpha']
    beta = params['beta']
    z = beta * z + alpha * g
    step = z

```

After calculating `step` at each step, all algorithms uniformly update parameters using $x = x - \text{step}$, and store the current position, loss value, and step size in an array for use in subsequent tasks.

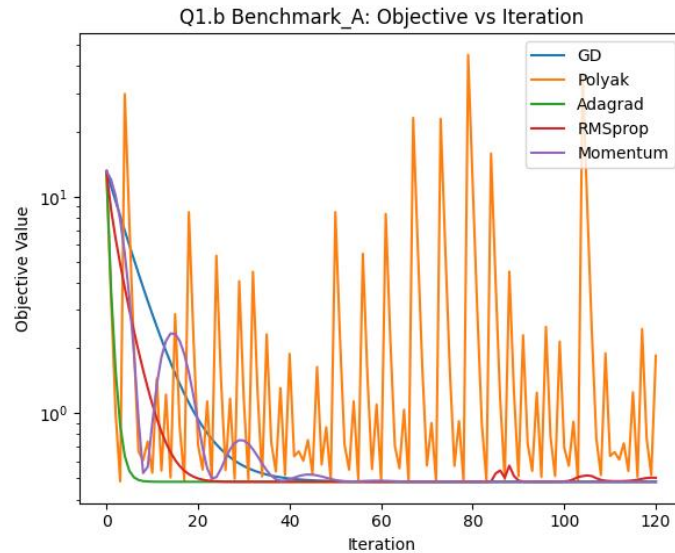
Q1. B

I tested and compared the five optimization algorithms (GD, Polyak, Adagrad, RMSprop, and Polyak momentum) implemented in the previous sub-question on three benchmark problems (A, B, C).

1. Benchmark A

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$$

The objective function is a convex quadratic function, with an overall shape resembling an elliptical bowl. Compared to other benchmarks, this function is relatively regular, and the terrain is not complex. Below is the convergence curve plot of the objective value versus the number of iterations.



Adagrad: Performed best. Since the loss function is a regular quadratic form, the adaptive learning rate quickly locks onto the optimal gradient direction, achieving near-perfect convergence within the first 5 iterations. It is very stable with no oscillations and remains stable at the optimal value throughout the subsequent 100+ iterations.

RMSprop: Performed second-best. During the first 20 iterations, RMSprop rapidly and smoothly descended to its lowest point of 0. However, it is worth noting that between iterations 80 and 120, the red line exhibits slight fluctuations. This is because RMSprop uses an exponentially weighted moving average, causing historical gradients to continuously decay. As the value approaches the local minimum, the smaller denominator causes the effective step size to occasionally be amplified again. Combined with minor numerical noise, this results in a slight rebound.

GD: Since the learning rate ($\alpha = 0.08$) is set conservatively and the objective function is highly regular, GD exhibits a very smooth and monotonically decreasing curve. Although its convergence speed is slower than the previous two algorithms, it demonstrates exceptional stability, reaching the minimum after 40 iterations and remaining stable thereafter.

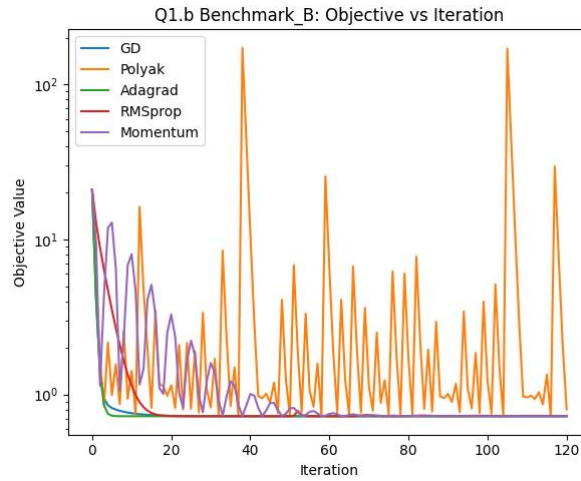
Momentum: There is significant fluctuation during the initial convergence phase, with noticeable large swings. This is the typical “overshoot” phenomenon caused by momentum accumulation. However, the overshoot gradually diminishes, and the algorithm reaches the minimum after 50 iterations, stabilizing at the local minimum.

Polyak Step Size: Performs the worst in this scenario and is highly unstable. Over 120 iterations, the function value fluctuates wildly throughout the process and ultimately fails to reach the minimum, remaining at a high value. Since its step size depends on $\|\nabla f(x_k)\|^2$, as the gradient becomes smaller near the optimal solution, the drastic changes in the denominator cause frequent oscillations.

2. Benchmark B

$$f(x_1, x_2) = (x_1 - 1)^2 + 5(x_2 - 2)^2 + \sin(x_1)$$

Benchmark B introduces a $\sin(x_1)$ term, giving it nonlinear and slightly non-convex characteristics. This implies that there will be slight undulations on the inner walls of the function, resulting in a more complex terrain and placing higher demands on the optimization algorithm. Below is the convergence curve of the objective value versus the number of iterations.



Adagrad and RMSprop: They remain in the lead. Even with nonlinear perturbations, the coordinate-wise adaptation mechanism still balances the update steps in the directions of x_1 and x_2 . Reaching the optimal point within the first 5 iterations, Adagrad is slightly faster than RMSprop. Both maintain stability throughout the convergence process without any oscillations.

GD: Its performance is nearly identical to Adagrad's. This is because, provided the learning rate is set appropriately, the perturbations in this problem are not sufficient to trap GD. It can smoothly traverse the shallow pits generated by $\sin(x_1)$ and exhibits good stability.

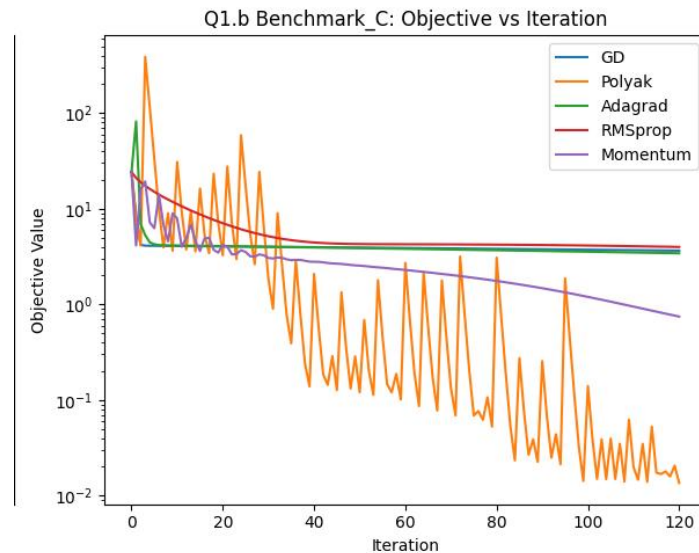
Momentum: Compared to Benchmark A, the momentum method exhibits longer-lasting fluctuations on Benchmark B. This indicates that on nonlinear surfaces, the “inertia” of momentum requires more iterations to offset the interference caused by nonlinear terms; however, around 60 iterations, Momentum also reaches the optimal solution and remains stable.

Polyak Step Size: Oscillations are extremely severe. As shown in the graph, the amplitude of these oscillations far exceeds that of Benchmark A. This reflects that when the objective function exhibits nonlinear fluctuations, the Polyak step size's sensitivity to the gradient is amplified, making the algorithm highly prone to significant oscillations within the solution space.

3. Benchmark C

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$$

The Rosenbrock function is the most complex of the three benchmarks. Its shape resembles an extremely narrow “banana-shaped” valley with steep walls. At the valley floor, pointing toward the optimal solution, the gradient is very small, creating highly pathological conditions. Below is the convergence curve plotting the objective value against the number of iterations.



Momentum: Best Performance. For this extreme function, the momentum method is the only algorithm that demonstrates a sustained, stable downward trend. Momentum helps the algorithm maintain balance amidst the steep gradients on the valley walls and continue advancing deeper into the valley. However, convergence is very slow.

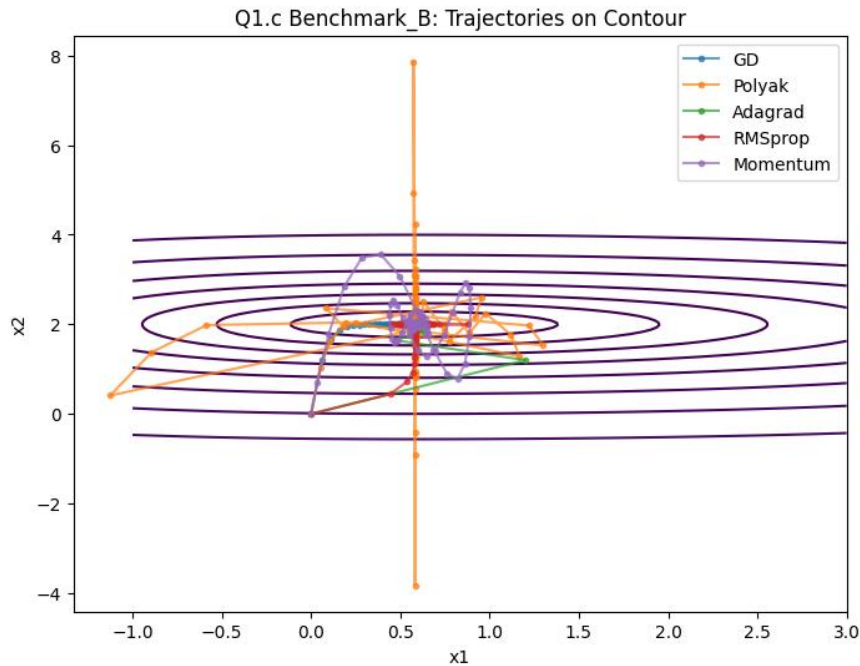
GD, Adagrad, and RMSprop: Although these three algorithms also achieved a steady descent, they quickly entered a plateau phase, stagnating between $10^1 - 10^0$ and appearing as a straight line. I believe this is due to the extremely pathological curvature characteristics of the Rosenbrock valley. GD's single scalar step size cannot accommodate both the steep sidewalls and the gentle valley floor; Adagrad accumulates a massive gradient squared during the initial descent through the canyon walls, causing the effective learning rate to decay prematurely toward zero and leading to a mid-course stagnation; while RMSprop, despite having a decay mechanism, exhibits extremely weak vertical progress along the gentle valley floor at the conservative learning rate set, resulting in all three algorithms becoming stuck. In terms of convergence speed, GD is roughly equivalent to Adagrad and faster than RMSprop.

Polyak Step Size: Although it continues to exhibit high-frequency oscillations, it is worth noting that in the later stages of iteration, the target value reached by the Polyak step size (approximately 10^{-2}) is significantly lower than that of the stagnating GD, RMSprop, and Adagrad (approximately 10^0). This suggests that in regions where optimization is difficult, the aggressive and unstable jumps characteristic of Polyak actually increase the probability of reaching deeper optimal regions.

Q1. C

The contour plots of the optimization trajectories for the five algorithms on benchmarks B and C are shown below.

1. Benchmark B



The contour lines for Benchmark B form a flattened elliptical shape. Since the coefficient in the x_2 direction is five times that of the x_1 direction, the surface is very steep in the vertical direction but relatively gentle in the horizontal direction.

Adagrad (green line): It follows a zigzag path, reaching the optimal solution rapidly after 3–4 steps. This confirms the mathematical properties of Adagrad's cumulative squared gradient: during the initial descent, the large gradient in the x_2 direction increases the denominator, causing the learning rate in the x_2 direction to decrease in the 2nd and 3rd steps, while the learning rate in the x_1 direction remains relatively constant. This results in a sharp turn in the trajectory during the early stages.

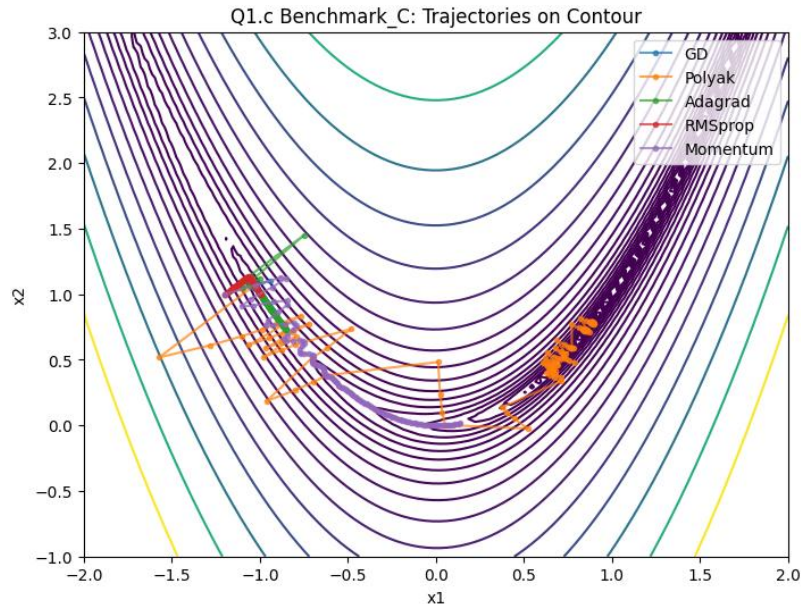
RMSprop (red line): Unlike Adagrad, the red line follows a smoother and more direct path as it enters the central region (the valley). However, it is worth noting that after reaching the central region, it undergoes a slight lateral movement along the x_1 axis. This is due to the presence of the decay rate, which allows it to "forget" the historical gradients in the x_2 direction. When it reaches the gentle valley bottom, the x_2 gradient becomes small, while the x_1 direction still retains a weak gradient due to the presence of $\sin(x_1)$, leading to this lateral movement along the x_1 axis.

Momentum (purple line): The trajectory exhibits a spiral-like behavior. Because the momentum term accumulates the velocity from earlier stages, when it rushes toward the local minimum, inertia causes it to overshoot. Subsequently, pulled by the gradient in the opposite direction, it turns back, eventually stabilizing at the center after several spirals, much like a steel ball rolling at the bottom of a bowl.

GD (blue line): It behaves in a straightforward manner, moving smoothly along the negative gradient direction perpendicular to the contour lines. Due to a moderate learning rate, no significant oscillations occur, and it glides along a curve to the lowest point.

Polyak Step Size (Orange Line): The trajectory is extremely chaotic. It makes huge, erratic jumps through space. This confirms the severe oscillations observed in the previous Task (b), indicating that its step size calculation is highly unstable on this terrain.

2. Benchmark C



The contour lines in Benchmark C form a long, banana-shaped valley. The walls on both sides of the valley are extremely steep, but the valley floor is extremely flat.

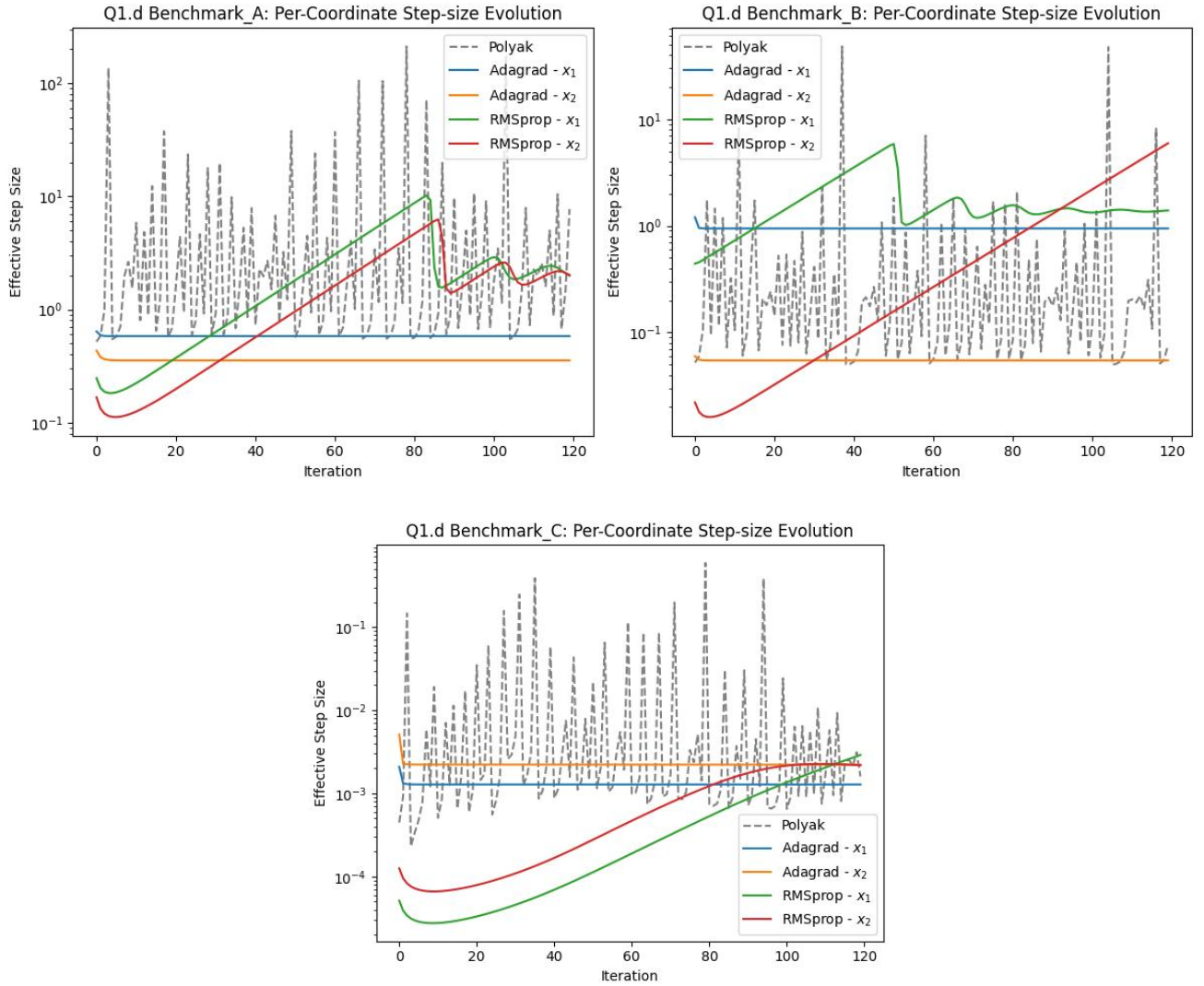
Momentum (Purple Line): In the banana-shaped canyon, the standard gradient is almost always perpendicular to the canyon walls on both sides. The momentum method initially exhibits pronounced zigzag bouncing along the canyon walls, but the amplitude of these bounces gradually decreases, while longitudinal momentum directed toward the depths of the valley floor is continuously accumulated. This allows the purple line to advance smoothly but slowly along the curved valley floor, making it the most stable algorithm among all.

GD (blue line), Adagrad (green line), and RMSprop (red line): After entering the valley floor, the trajectories of these three algorithms rapidly converge. Due to the extremely small gradient at the valley floor, the step sizes of GD and RMSprop are insufficient to sustain their progress; meanwhile, Adagrad accumulates too much historical gradient during the initial descent, causing the learning rate to approach zero at this point and the algorithm to come to a complete standstill.

Polyak step size (orange line): Polyak's trajectory exhibits significant lateral bouncing between the two walls of the valley. Although this bouncing is highly unstable, it is precisely these large-stride jumps in the latter half of the valley that allow it to reach a deeper region than Momentum. It is the algorithm that comes closest to the optimal solution among all algorithms.

Q1. D

Plot the variation of effective step size versus the number of iterations for the Polyak, Adagrad, and RMSprop algorithms on benchmarks A, B and C. The results are as follows.



Polyak Step Size: In all three benchmarks, the Polyak step size (gray dashed line) exhibits severe high-frequency oscillations. This is because Polyak's update formula is $\alpha_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2 + \epsilon}$. When the algorithm enters a region of low gradient, $\|\nabla f(x_k)\|^2$ becomes extremely small, and even minor numerical perturbations can cause the step size to be amplified. This lack of stability leads to the chaotic trajectories observed in tasks (b) and (c).

Adagrad: In the three plots, Adagrad's effective step size x_1 (blue line) and x_2 (orange line) exhibit two clearly distinct and stable horizontal lines. Taking Benchmark B as an example, since the objective function is extremely steep in the direction of x_2 , the massive gradient generated by x_2 during the first iteration is accumulated in the denominator, causing the effective step size to drop and remain at an extremely low level; whereas the step size in the direction of x_1 is relatively large. This separation demonstrates its ability to handle feature scale imbalance.

After an extremely brief initial descent, Adagrad's step size curve becomes almost completely horizontal. This is because, in benchmarks A and B, the algorithm converges within the first few steps, the gradient approaches 0, and the denominator no longer increases; whereas in benchmark C, the algorithm gets stuck in a shallow valley, where even extremely small gradients can no longer drive the denominator to increase, causing the step size to remain unchanged. This explains why Adagrad stalls in benchmark C.

RMSprop: The step size curve of RMSprop demonstrates the most complex dynamic adaptation process. Unlike Adagrad, the effective step size of RMSprop shows a distinct and

sustained increase during the early to mid-stages of iteration. For different terrains, the trend of step size changes varies in the mid-to-late stages of iteration.

The rapid rise in the early stages is due to the algorithm encountering large descent gradients at the beginning of the iteration, causing the denominator $\sqrt{E[g^2] + \epsilon}$ to increase and the step size to be compressed. However, as the algorithm quickly enters a flat region, the damping effect of β begins to manifest: it continuously forgets the historical gradients from the early stages. The denominator continues to shrink, and the corresponding step size α_k increases.

The trend of step size changes differs across benchmarks:

Benchmark A and Benchmark B: In these two problems, because the algorithm can quickly approach the optimal solution region, the large gradients from the early stages are rapidly forgotten, causing the green and red lines to rise rapidly in the early to mid-stages. Particularly in Benchmark B, the green line peaks around the 50th iteration and then drops precipitously. This corresponds to the algorithm encountering a slight steep slope generated by $\sin(x_1)$ while moving horizontally at the valley floor; the algorithm reduces its step size to prevent oscillation, demonstrating strong dynamic adjustment capabilities.

Benchmark C: In the Rosenbrock Canyon, the algorithm fell into the gentle valley floor during the first few iterations and remained in this low-gradient environment throughout the early, middle, and late stages. Consequently, after an initial surge, the historical gradient of RMSprop entered a period of decay. This is reflected in the graph as a gradual ascent with nearly constant slope throughout the early, middle, and late stages, following the initial 10 descents. However, despite this ascent, the algorithm stalled in the Rosenbrock valley in Question C because the gradient was too small and the step size insufficient at that point.

Q1. E

Based on the aforementioned comparative analysis of the target value descent curves, contour line trajectories, and adaptive step size evolution, I have summarized the overall performance of the five optimization algorithms on the three benchmark problems in the table below:

Algorithm	Speed	Smoothness	Stability	Limitations
Gradient Descent (GD)	Moderate	Very high	Extremely high	Cannot handle inconsistencies in scaling across different dimensions; highly prone to oscillation or stagnation in pathological valleys (Benchmark C)
Polyak step size	Fast (but highly unpredictable)	Very low (severe and frequent spikes)	Very low	Step size is sensitive to the global gradient norm, causing oscillations when approaching extrema or in nonlinear regions
Adagrad	Very fast (initial burst)	High (approaches in a zigzag)	Strong	The infinite accumulation of historical squared gradients causes the step size to vanish

Algorithm	Speed	Smoothness	Stability	Limitations
		pattern)		prematurely, making it prone to stagnation in long, flat regions (Benchmark C)
RMSprop	Extremely fast (continuous acceleration)	Extremely high (with slight adjustments later on)	Extremely strong	Constrained by the minimum constant ϵ , at the bottom of an extremely pathological region where the gradient is nearly zero, the step size recovery reaches an extreme value, causing the actual physical update to stall (Reference C)
Polyak momentum	Fairly fast (robust acceleration)	Moderate (may overshoot initially)	Extremely strong	Due to inertia, slight oscillations may occur near local minima (Reference B)

Based on the analysis of the previous tasks, we can conclude that:

Gradient descent (GD) with a constant step size is too conservative to handle anisotropic curvature changes in high-dimensional spaces. Although the Polyak step size attempts dynamic adjustment, it is extremely unstable due to its complete reliance on the global gradient scalar.

In contrast, adaptive algorithms (Adagrad and RMSprop) perform well when dealing with inconsistent feature scaling (Benchmarks A and B). However, Adagrad's monotonically decreasing learning rate causes it to stall prematurely when facing long-distance optimization paths (such as the valley in Benchmark C). By introducing an exponentially weighted moving average, RMSprop successfully achieves dynamic step size recovery, making it the most robust local adaptive algorithm on standard non-convex terrain.

Finally, when faced with the extremely ill-posed, narrow, and flat-bottomed Rosenbrock canyon (Benchmark C), adaptive algorithms relying solely on amplifying local gradients all failed, while the Polyak momentum method, leveraging its physical inertia mechanism that accumulates historical update directions, successfully counteracted lateral oscillations and became the only algorithm capable of continuously advancing deep into the canyon.

Q2

Q2. A, B

As required by the problem statement, I implemented the Nesterov and Adam optimization algorithms for benchmarks A, B, and C, and included a constant-step gradient descent (GD) as a comparison benchmark.

1. Nesterov

Nesterov is an improvement over the Heavy Ball algorithm from Problem 1. Unlike the fixed β in Problem 1, Nesterov's momentum weight gradually increases with the number of iterations k , controlled by the formula $\frac{k-1}{k+2}$, until it reaches an upper limit. This makes the algorithm more cautious in the initial stages and faster in the later stages.

It also introduces a look-ahead mechanism. It first takes a step forward by inertia, as if with its eyes closed, and then checks the gradient at that future position. If it detects a steep slope ahead, the gradient at the future position immediately provides a counterforce, effectively applying the brakes when z is actually updated.

Its core formula is:

$$z_{k+1} = \beta_k z_k - \alpha \nabla f(x_k + \beta_k z_k)$$

$$x_{k+1} = x_k + z_{k+1}$$

In the code, we break this down into three steps:

```
elif name == 'Nesterov':
    alpha = params['alpha']
    beta_max = params['beta_max']
    # Momentum coefficient
    beta_k = min((k - 1) / (k + 2), beta_max)
    # Look ahead with momentum
    x_lookahead = x + beta_k * z
    g_lookahead = grad_fn(x_lookahead)
    # Update momentum and position
    z = beta_k * z - alpha * g_lookahead
    x = x + z
```

2. Adam

Adam combines momentum with RMSprop and incorporates a bias correction mechanism. It uses two variables, m_t and v_t , to remember the direction and squared magnitude of the gradient.

In the early stages of iteration, since m_t and v_t are both initialized to the zero vector, updates can be extremely slow or misdirected. Dividing by $(1 - \beta^k)$ amplifies m_t and v_t in the beginning, eliminating the bias caused by their initial zero values. As K increases, the denominator approaches 1, and the bias correction term automatically fades out.

```
elif name == 'Adam':
    alpha = params['alpha']
    beta1 = params['beta1']
    beta2 = params['beta2']
    eps = params['eps']

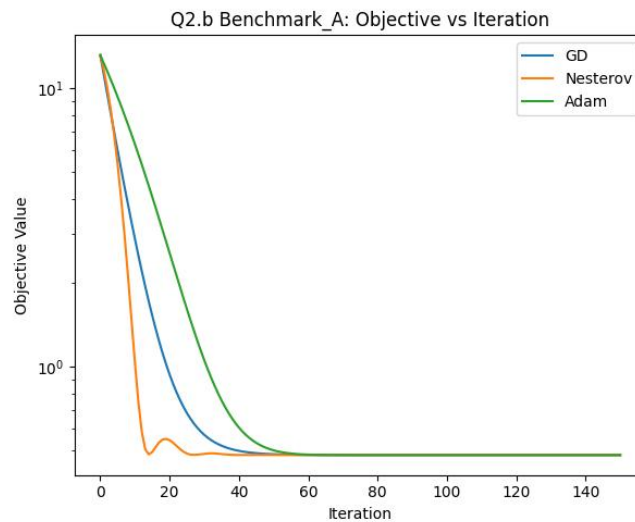
    g = grad_fn(x)
    m_t = beta1 * m_t + (1 - beta1) * g
    v_t = beta2 * v_t + (1 - beta2) * (g ** 2)
    # Bias correction
    m_hat = m_t / (1 - beta1 ** k)
    v_hat = v_t / (1 - beta2 ** k)

    step = alpha * m_hat / (np.sqrt(v_hat) + eps)
    x = x - step
```

I conducted tests and comparisons on three benchmark problems (A, B, and C). The three benchmark functions have already been analyzed in Q1, so I will not elaborate further here.

The convergence curves of the target value versus the number of iterations are shown below.

1. Benchmark A

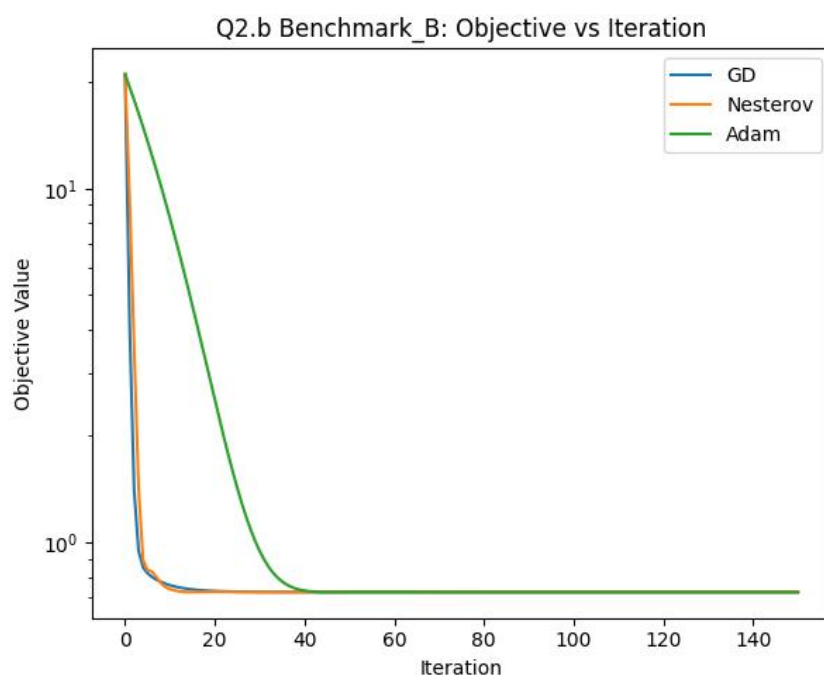


Nesterov: Nesterov converged the fastest, reaching the minimum within the first 10 iterations. However, overshooting occurred during the subsequent 10 iterations. This indicates that the accumulation of momentum significantly accelerated convergence, while its look-ahead mechanism pulled back the overshoot updates, ultimately stabilizing completely around 30 iterations.

GD: As a baseline, GD exhibits a very smooth, monotonically decreasing trajectory, converging at approximately 50 iterations.

Adam: Surprisingly, under these parameter settings, Adam performed the worst. Although it achieved an extremely smooth descent, its overall convergence rate was the slowest, converging after approximately 50 iterations. This suggests that its second-order momentum estimate may have imposed excessive penalties on the step size in this case.

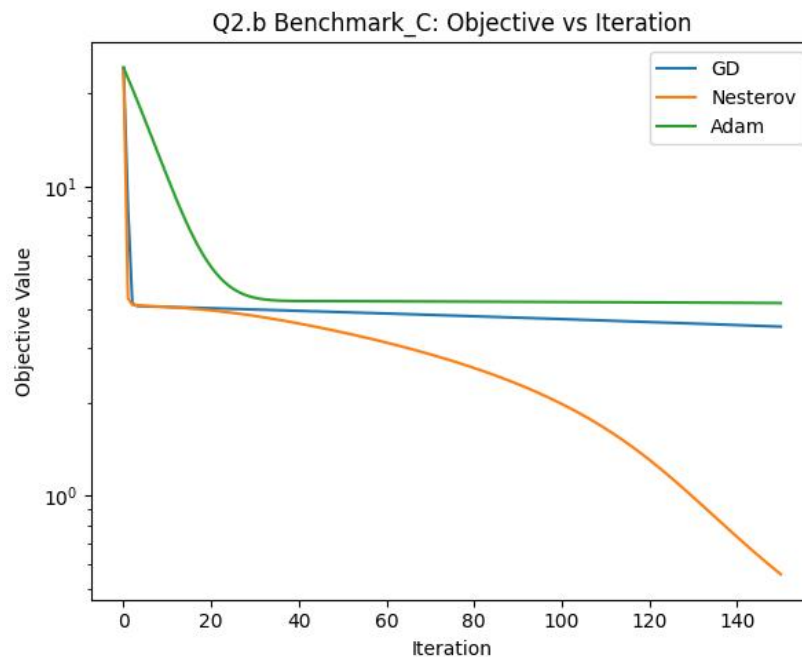
2. Benchmark B



GD and Nesterov: On this terrain, the trajectories of GD and Nesterov almost completely overlap, plunging vertically to near the optimal value within a very short time (the first 10 iterations). Nesterov exhibited only a very slight fluctuation at the bottom before quickly stabilizing, performing very well.

Adam: Similar to Benchmark A, Adam's convergence curve remains a smooth, oscillation-free descent, but it remains the slowest, not fully converging until around 40 iterations. This further confirms that, under the parameter settings of this problem, Adam tends to adopt a more conservative strategy.

3. Benchmark C



Nesterov: Under extreme function conditions, Nesterov demonstrated significant advantages. In the initial stages, the function drops rapidly along a vertical line within just a few steps. Unlike other algorithms, it does not stall afterward but maintains a continuous, steep descent. This is because Nesterov's dynamic momentum continuously accumulates vertical velocity along the valley floor, while its look-ahead mechanism allows it to anticipate the curvature of the valley in advance, thereby avoiding the loss of kinetic energy caused by hitting a wall.

GD: Due to a lack of momentum, GD can only move slowly at the bottom of the valley by relying on extremely weak local gradients, effectively coming to a standstill.

Adam: Adam continues to perform the worst. After a few initial rapid descents, the curve becomes a completely horizontal line, stalling earlier and at a higher elevation than GD. This exposes a fatal weakness of adaptive algorithms in this terrain: the massive historical squared gradients accumulated during the initial descent cause their effective learning rate to be compressed at the valley floor, while their limited first-order momentum is insufficient to provide the inertia needed to traverse the flat regions, ultimately leading to algorithmic stagnation.

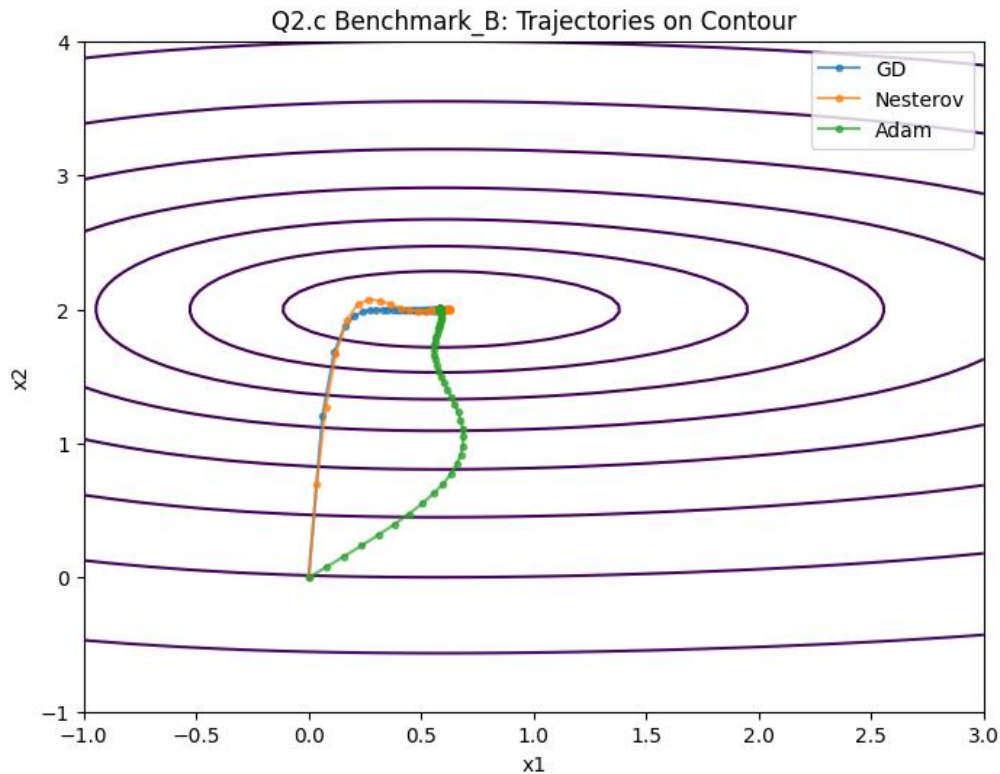
Based on the comprehensive experimental results, Nesterov performed best; its momentum mechanism can overcome pathological curvature, allowing it to maintain a certain speed even under minimal gradients. While Adam's descent is extremely smooth,

without careful hyperparameter tuning, its speed lags behind, and it loses its updating momentum prematurely in extremely flat valleys.

Q2. C

The contour plots of the optimization trajectories for the three algorithms on benchmarks B and C are shown below.

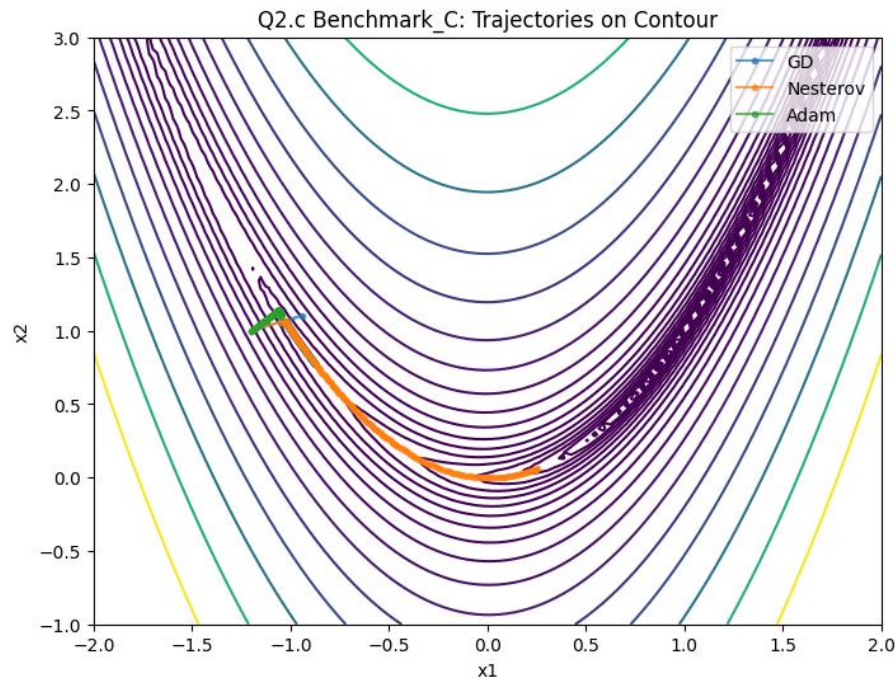
1. Benchmark B



Adam (green line): Adam exhibits a smooth curve distinct from the other two. The trajectories of the other two algorithms first enter the valley and then move toward the minimum, but Adam first moves diagonally for a distance, aligning the x -coordinate with the minimum before moving toward the valley. Because its denominator penalizes large gradients in the x_2 direction and compensates for weak gradients in the x_1 direction, Adam exhibits an upward-rightward bias at the start. It only enters the valley afterward.

GD (blue line) and Nesterov (orange line): The curves for these two algorithms almost completely overlap. Since both algorithms use a global scalar learning rate, they are initially guided entirely by the steepest gradient in the x_2 direction, resulting in a nearly vertical downward trajectory. Only after reaching the valley floor do they begin to move laterally along the x_1 direction. It is worth noting that, compared to the standard Polyak momentum in Problem 1, the Nesterov momentum here produces almost no excess oscillation or overshoot after the turn. This demonstrates the braking capability of the look-ahead mechanism.

2. Benchmark C



Nesterov (orange line): Nesterov performs best here. Its trajectory not only enters the valley rapidly but also, at the valley floor where the gradient is very small, conforms extremely smoothly to the valley's curvature, moving gently toward the minimum without any oscillations. Although it did not reach the minimum in the end, it is the algorithm that came closest to the minimum among all algorithms.

Adam (green line): Although Adam possesses momentum, it came to a complete standstill after a rapid initial descent, appearing as a green dot corresponding to the horizontal line in 2.b. The second-order momentum v_t accumulated while descending the steep valley walls caused its effective step size to become extremely small upon entering the gentle valley floor, leading to stagnation.

GD (blue line): As expected, GD's scalar step size lost momentum on the extremely gentle and curved terrain, resulting in stagnation.

Q2. D

In this task, I implemented two batch sizes of Mini-Batch SGD, applied them to Benchmark A, and conducted a comparison.

In the implementation, before sampling at the start of each epoch, I used `np.random.permutation` to randomly shuffle the indices of the 1,000 samples, then sliced the shuffled dataset sequentially according to the specified batch sizes (5, 40). I calculated the local gradients $g = \frac{1}{b} X_{\text{batch}}^T (X_{\text{batch}} \theta - y_{\text{batch}})$ and performed parameter updates.

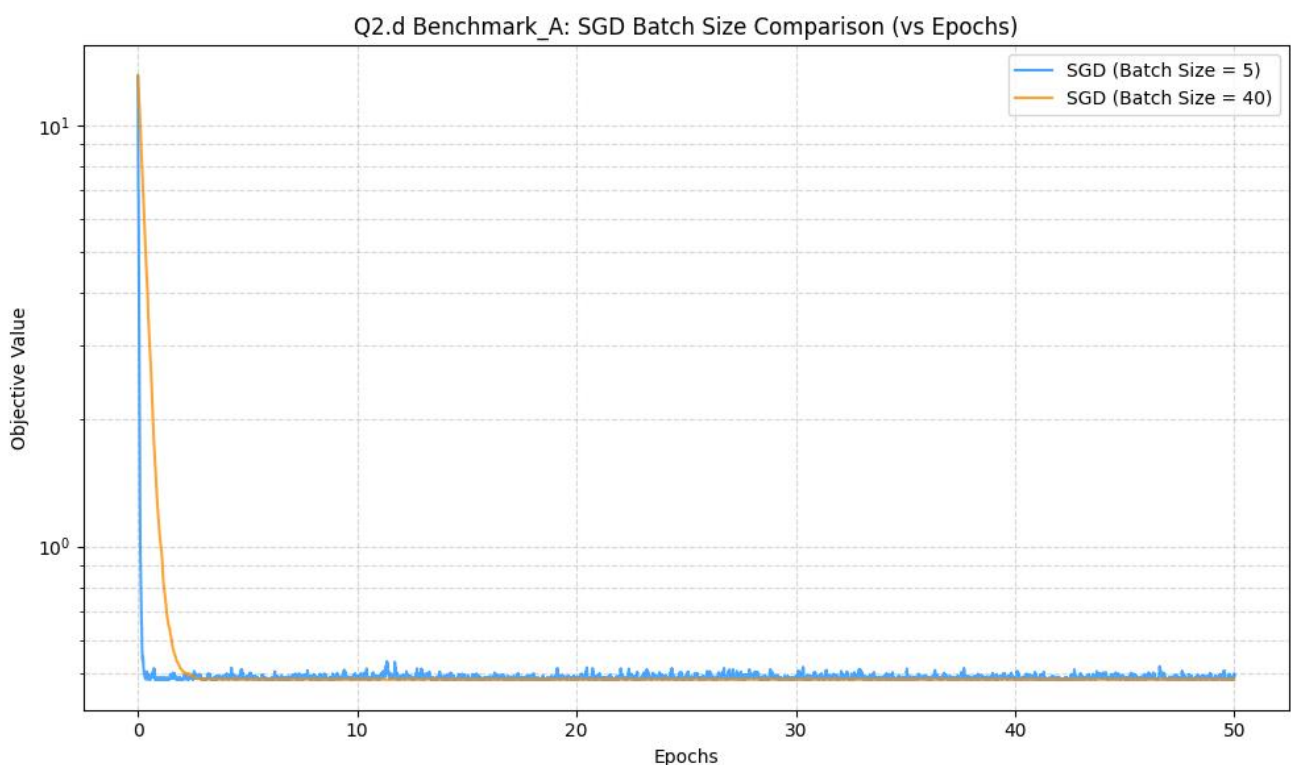
```
def run_sgd(X, y_target, theta0, alpha, batch_size, epochs):
    m_samples = X.shape[0]
    theta = np.array(theta0, dtype=float)
    # Compute full-batch loss
    history_loss = [0.5 * np.mean((X @ theta - y_target)**2)]
    for epoch in range(epochs):
        # Shuffle the data at the start of each epoch
        indices = np.random.permutation(m_samples)
```

```

X_shuffled = X[indices]
y_shuffled = y_target[indices]
# Iterate over all mini-batches
for i in range(0, m_samples, batch_size):
    Xi = X_shuffled[i:i+batch_size]
    yi = y_shuffled[i:i+batch_size]
    # Compute batch gradient
    grad = Xi.T @ (Xi @ theta - yi) / batch_size
    theta = theta - alpha * grad
    # Record full-batch loss after each update
    history_loss.append(0.5 * np.mean((X @ theta - y_target)**2))
return np.array(history_loss)

```

Applying this to Benchmark A, both were run for 50 epochs with a learning rate of $\alpha = 0.06$. The target-Epochs convergence curves are shown below.

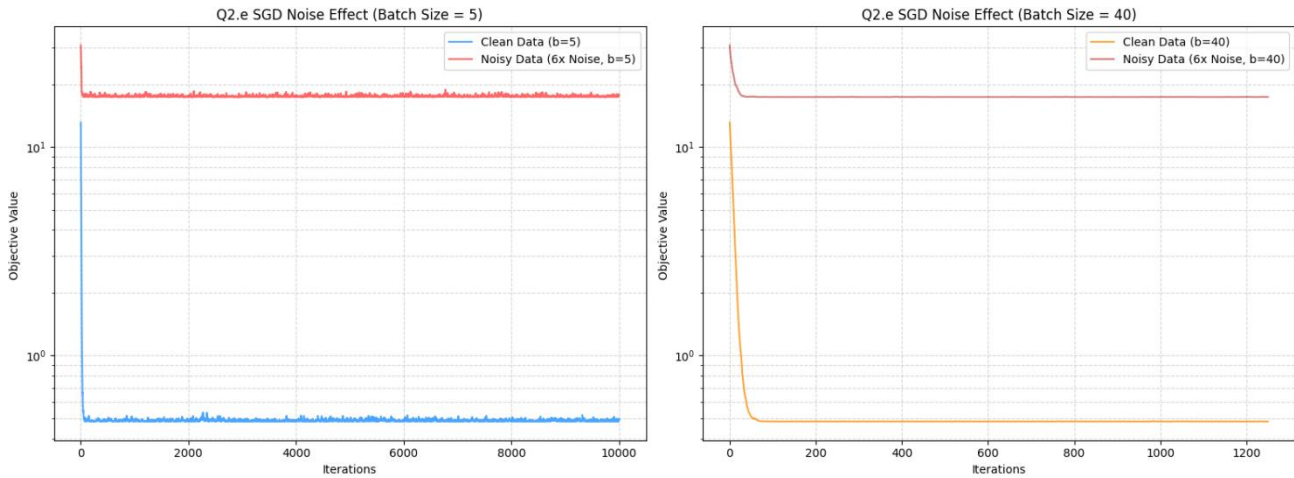


Small batch size (b=5): The blue line rapidly approaches the minimum during the first few epochs. After reaching the vicinity of the minimum, it exhibits slight high-frequency oscillations and fails to stabilize completely at a fixed point. This is because estimating the gradient of the entire dataset using only 5 samples results in significant error, causing the algorithm to continuously oscillate near the optimal point.

Large batch size (b=40): In contrast, the orange line is extremely smooth at the bottom, almost becoming a straight line. Increasing the batch size significantly reduces the variance in gradient estimates, causing each update to more closely approximate the true global gradient direction, thereby achieving more stable and precise convergence. However, observing the initial descent phase of the curves reveals that the blue line achieves a cliff-like drop to the target value within a very small number of updates. This indicates that larger batch sizes are more stable, while smaller batch sizes are faster.

Q2.E

For this task, as required by the problem statement, I injected noise equivalent to six times the original level into the target variable y of Benchmark A, and ran SGD with batch sizes of $b=5$ and $b=40$, respectively. The comparison plots of function values versus the number of updates are shown below.



In both the " $b = 5$ " (left graph) and " $b = 40$ " (right graph) plots, the noise data curve shows a significantly elevated final convergence value. This is because noise cannot be fitted by any linear model. In this experiment, by increasing the noise, we effectively raised the function's theoretical optimal solution.

Small Batch ($b = 5$): In the left figure, the red line exhibits slight high-frequency oscillations after reaching the trough. This is because data noise further amplifies the statistical variance of the small-batch gradient estimate. Sampling only 5 samples at a time makes it highly likely to draw contaminated data, causing the gradient direction to deviate.

Large-batch($b = 40$) : In the right figure, the brown line also converges to a higher loss value, but its curve is very smooth. This demonstrates that increasing the batch size is an extremely effective means of variance reduction. By averaging the gradients of 40 samples at once, the random noise in individual samples is internally averaged out. Even when dealing with high-noise datasets, large-batch SGD remains very stable.

Q3

Q3. A

Given the function $g(x) = x^4$, construct its first- and second-order local approximations at the expansion point $x_0 = 0.25$.

First, compute the function value at this point, along with its first- and second-order derivatives:

$$\text{Function value: } g(0.25) = (0.25)^4 = 0.00390625$$

$$\text{First derivative: } g'(x) = 4x^3 \Rightarrow g'(0.25) = 0.0625$$

$$\text{Second-order derivative: } g''(x) = 12x^2 \Rightarrow g''(0.25) = 0.75$$

First-order Local Approximation: Using the first-order Taylor expansion formula $g(x) \approx g(x_0) + g'(x_0)(x - x_0)$, we obtain:

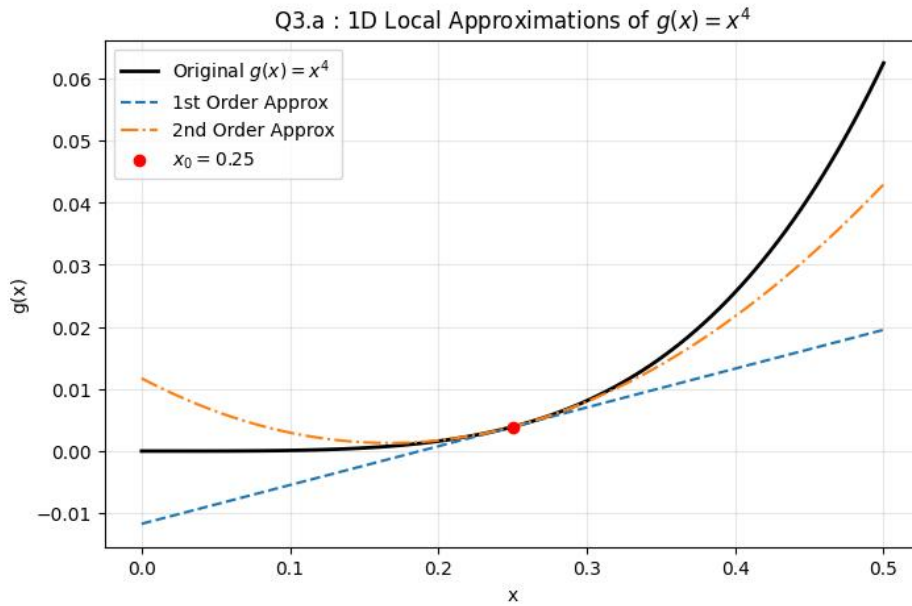
$$g_{1st}(x) = 0.00390625 + 0.0625(x - 0.25)$$

Second-order Local Approximation: Using the second-order Taylor expansion

formula $g(x) \approx g(x_0) + g'(x_0)(x - x_0) + \frac{1}{2}g''(x_0)(x - x_0)^2$, we obtain:

$$g_{2nd}(x) = 0.00390625 + 0.0625(x - 0.25) + 0.375(x - 0.25)^2$$

The graph of the function is shown below



First-order approximation (blue dashed line): Essentially, this is the tangent line at x_0 . It reflects the slope of the function at that point, but deviates significantly from the original function curve as it moves away from that point.

Second-order approximation (blue solid line): Essentially a parabola. Because it incorporates the second derivative, which contains information about curvature, this quadratic curve fits the original function within a fairly wide neighborhood of x_0 .

Q3. B

The core of the Newton method lies in using the Hessian matrix to calculate the curvature of the current terrain. When dealing with complex terrain such as Reference C, the diagonal elements of the Hessian matrix $\left(\frac{\partial^2 f}{\partial x_i^2}\right)$ measure the steepness in a single dimension. On steep canyon walls, the curvature is extremely high, while in flat valley bottoms, the curvature is extremely low. The off-diagonal elements of the Hessian matrix, $\left(\frac{\partial^2 f}{\partial x_i \partial x_j}\right)$, measure the mutual influence between x_1 and x_2 .

In the Newton's method update formula, multiplying by H^{-1} is equivalent to performing an adaptive geometric distortion on the parameter space. It automatically reduces the step size in steep directions and increases it in flat directions, forcibly twisting the gradient direction—which would otherwise head toward the sidewalls—to follow the valley floor.

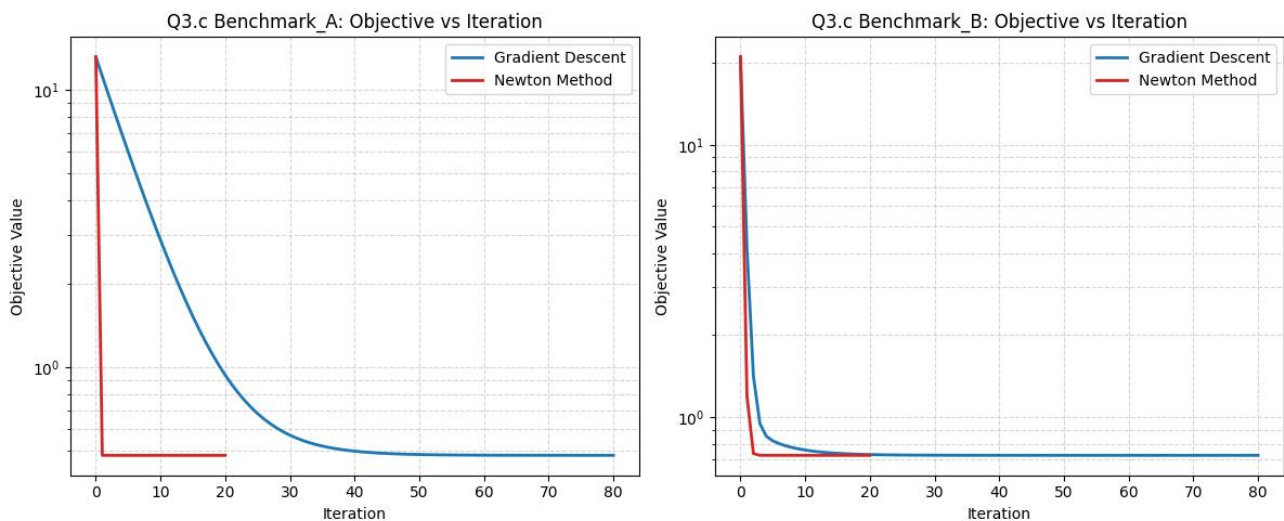
In the code implementation ($H_damped = H + damping * I$), I added a damping term (10^{-8}) as required. This is because in extremely flat regions (where curvature approaches 0), pure geometric distortion would be infinitely amplified, causing the algorithm to collapse or reverse direction. Adding this damping is equivalent to adding a small basic gradient, ensuring the algorithm's stability in flat regions.

```
def run_newton(loss_fn, grad_fn, hessian_fn, x0, alpha, iters, damping=1e-8):
    x = np.array(x0, dtype=float)
    h_x, h_loss, h_update = [x.copy()], [loss_fn(x)], []
    I = np.eye(len(x0))

    for _ in range(iters):
        g = grad_fn(x)
        H = hessian_fn(x)
        # Damped Hessian for numerical stability
        H_damped = H + damping * I
        d = np.linalg.solve(H_damped, -g)
        step = alpha * d
        x = x + step
        h_x.append(x.copy())
        h_loss.append(loss_fn(x))
        h_update.append(np.linalg.norm(step))
    return np.array(h_x), np.array(h_loss), np.array(h_update)
```

Q3. C

Apply the implemented Newton's method to benchmarks A, B, and C, compare them with GD, and plot the convergence curves of objective value versus number of iterations. The results are shown below.



1. Benchmark A

In Figure A, the Newton method (red line) exhibits a steep, cliff-like drop in the first iteration, reaching the optimal solution immediately. In contrast, GD (blue line) shows a typical slow, smooth decline, taking approximately 40 iterations to reach the minimum.

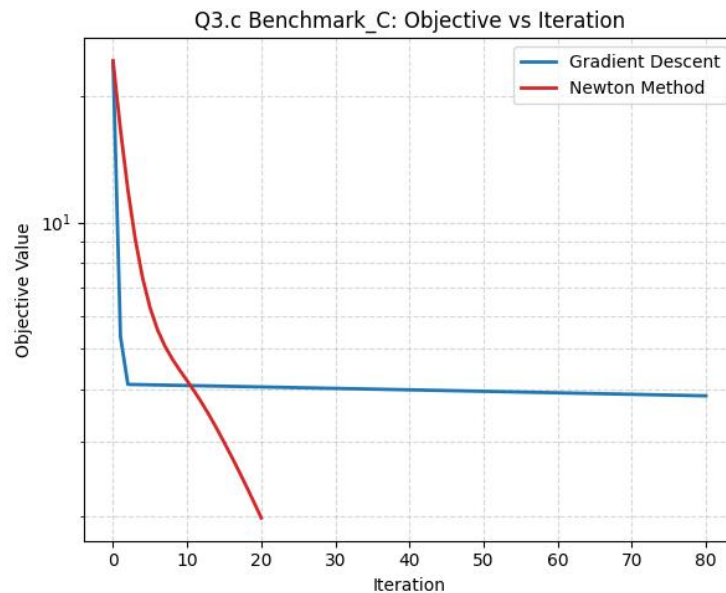
This is because the MSE loss function in linear regression geometrically corresponds to a quadratic parabolic surface. Since the Newton method fits the current terrain using a quadratic surface, its local fit in Benchmark A matches the actual terrain. Consequently, the Hessian matrix can directly point to the global optimal solution in a single step.

2. Benchmark B

In Figure B, despite the addition of nonlinear “ $\sin(x_1)$ ” perturbations to the terrain and the fact that the steepness in the “ x_2 ” direction is five times that of “ x_1 ,” the Newton method still

demonstrates a speed advantage, converging completely within 1 to 2 steps, slightly faster than GD. GD performs reasonably well here because the problem is sufficiently simple; however, it still requires over a dozen iterations to adjust the step sizes across different dimensions. In contrast, the Newton method reaches the optimal solution region in very few steps.

3. Benchmark C



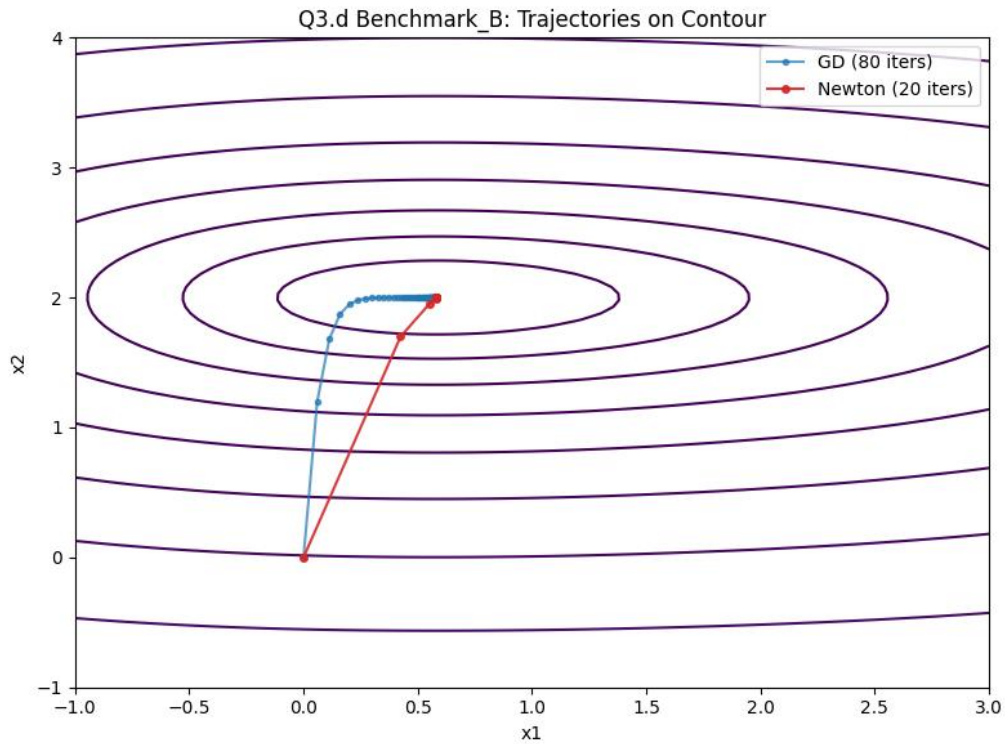
In Figure C, after a very brief initial descent, GD (blue line) completely flattens into an almost horizontal line, effectively stalling. In contrast, the Newton method (red line) faces no obstacles, maintaining an extremely steep slope as it continues toward the minimum.

The above experiments demonstrate that the Newton method has significant advantages over gradient descent in terms of iterative efficiency and adaptability to ill-conditioned terrain.

Q3. D

The contour plots of the optimization trajectories for the three algorithms on benchmarks B and C are shown below.

1. Benchmark B

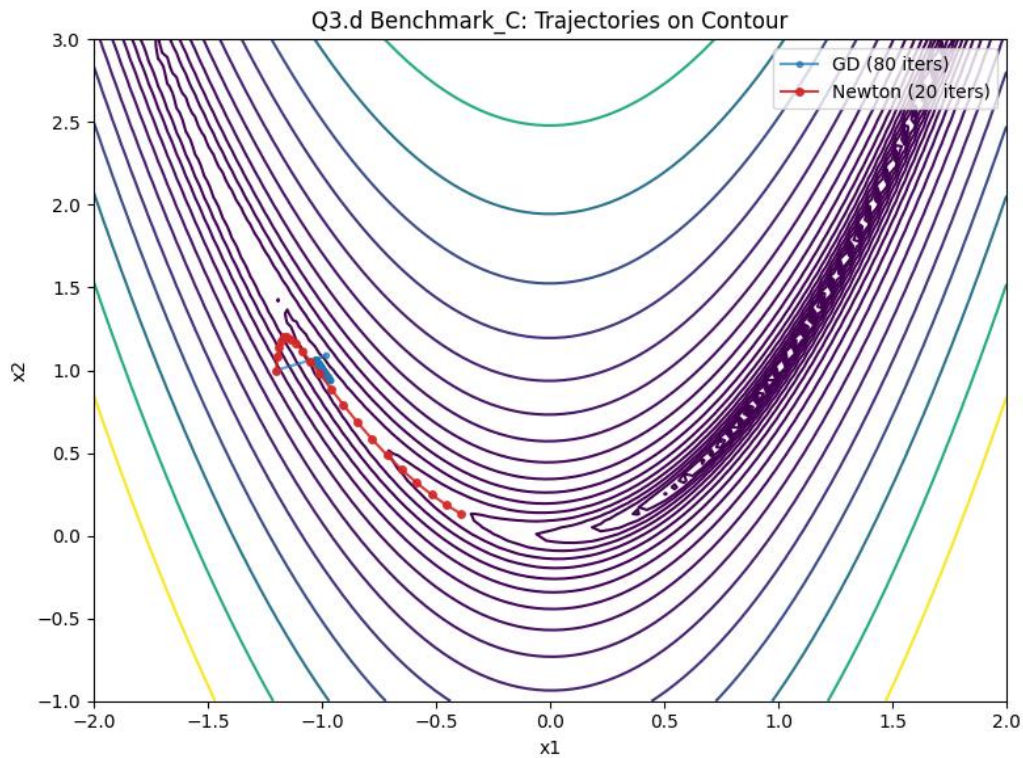


2.

GD (blue line): As clearly visible from the trajectory, the blue line drops almost vertically at the start, rapidly reaching the valley floor. This is because the gradient in that dimension is extremely large, completely dictating the initial direction of movement. Only after reaching the valley floor does GD begin to move slowly to the right along the x_1 axis.

Newton (red line): The Newton method follows a nearly straight line pointing directly toward the optimal solution. By multiplying by the inverse of the Hessian matrix, the Newton method stretches these distorted ellipses into perfect circles. The direction of the optimal solution points directly to the center of the circle, reaching the optimal solution in just two or three steps.

2. Benchmark C



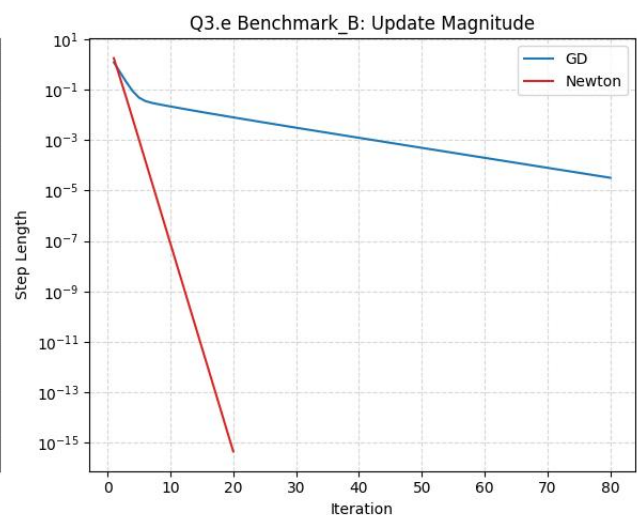
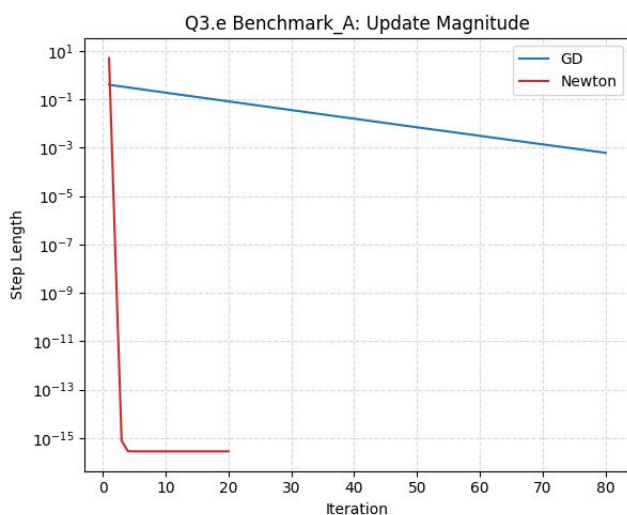
GD (blue line): After entering the valley, the blue trajectory stalls and appears as a cluster of blue dots.

Newton (red line): After rapidly entering the valley floor, it precisely follows the curved path of the valley bottom, moving very smoothly without any oscillations. This demonstrates that the off-diagonal elements of the Hessian matrix accurately measure the relationship between x_1 and x_2 . The Newton method not only recognizes that the terrain is currently flat but also anticipates an upcoming turn, maintaining a relatively fast pace even on the flat valley floor.

Experiments demonstrate that the Newton method performs exceptionally well, achieving adaptability to extreme functions.

Q3. E

The update magnitude versus iteration count curves for GD and the Newton method are shown below.



1. Benchmark A

GD (blue line): The update magnitude of GD forms a stable, downward-sloping straight line with a constant gradient on a logarithmic scale. The error shrinks by a fixed proportion at each step, resulting in a slow convergence process.

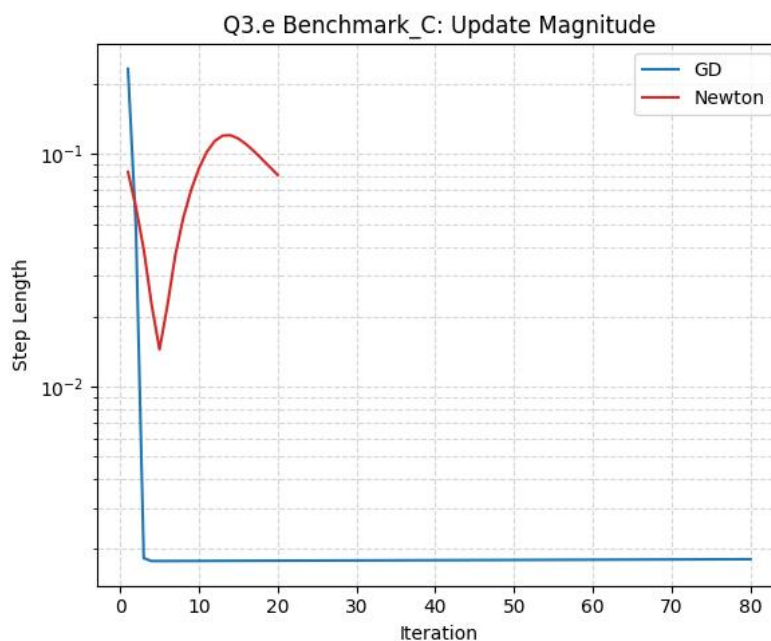
Newton (red line): Between steps 1 and 2, the step size drops sharply from the order of magnitude of 10^{-1} to a minimum below 10^{-15} . Combined with the trajectory plot for task d, this indicates that the Newton method reaches the optimal solution extremely quickly and remains stable thereafter.

2. Benchmark B

GD (blue line): Due to the imbalance between the X_1 and X_2 dimensions, GD rapidly traversed the steep gradient in the initial phase and immediately entered a period of extremely slow step size decay. Its movement along the flatter dimensions is very slow.

Newton (red line): It appears as a straight diagonal line, with the step size rapidly dropping to the order of 10^{-15} after 20 iterations. This demonstrates that the Hessian matrix successfully mitigates the resistance caused by anisotropy at the underlying level; once the algorithm enters the region near the optimal solution, it immediately triggers a secondary convergence mechanism to approach the target.

3. Benchmark C



GD (blue line): The blue line experiences a cliff-like drop within a very short number of iterations, but this is not because it has converged; rather, it has stalled. At the flat bottom, the gradient is extremely small, causing GD's actual step size to shrink to a level below 10^{-2} , leaving it stuck in place.

Newton (red line): In contrast, the Newton method's step size not only avoids falling into the valley but, after a brief dip, rises back to the 10^{-1} range, remaining there until the 20th step. This means that the Newton method maintains a large step size even at the flat bottom of the valley. This is because the inverse of the Hessian matrix adaptively amplifies the step size along extremely flat directions. This allows the Newton method to maintain an extremely high rate of progress.

Experiments demonstrate that the step size of gradient descent is entirely constrained by the current terrain slope; once the slope flattens, the algorithm faces stagnation. In contrast, the Newton method relies on the inverse of the curvature to distort space, shortening its

steps on steep terrain and increasing them on flat terrain. This adaptive mechanism allows the Newton method to converge rapidly while ignoring pathological terrain.

Q4

Q4.A, B

In accordance with the problem statement, I implemented two optimization algorithms: forward finite differences and Nesterov stochastic gradient descent, and included a constant-step gradient descent (GD) algorithm as a baseline for comparison.

Below are the implementations of each algorithm:

1. Forward Finite Difference

Forward finite differences take an extremely small step forward along each coordinate axis and calculate the partial derivative of that dimension by measuring the change in the function value. The corresponding formula is:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \delta e_i) - f(x)}{\delta}$$

In the code implementation, I iterate through each dimension using a loop. For the current dimension, I use `e = np.zeros\_like(x)` combined with `e[i] = 1.0` to construct the unit vectors along the coordinate axes `e\_i`, and then substitute these into the formula to compute the gradient.

```
def finite_difference_grad(loss_fn, x, delta):
    grad = np.zeros_like(x)
    for i in range(len(x)):
        e = np.zeros_like(x)
        e[i] = 1.0
        grad[i] = (loss_fn(x + delta * e) - loss_fn(x)) / delta
    return grad
```

2. Nesterov Stochastic Search

Unlike forward finite differences, which explore along fixed coordinate axes, Nesterov stochastic search involves selecting a random direction at the current point and taking a small step forward in that direction. It determines parameter updates by measuring the steepness of the gradient in that direction. The corresponding formula is:

$$x_{k+1} = x_k - \alpha \left[\frac{f(x_k + \delta u_k) - f(x_k)}{\delta} \right] u_k$$

In implementation, the code first generates a random number from a normal distribution, then normalizes it by dividing by its modulus to uniformly sample a random direction vector u_k on a multidimensional sphere. Unlike finite differences, which require n calculations, stochastic search performs a single trial along the newly generated direction u_k . If the gradient is downward (negative gradient), the update proceeds along u_k ; if the gradient is upward (positive gradient), the update moves backward along u_k .

```
def run_nesterov_rs(loss_fn, x0, alpha, delta, iters):
    np.random.seed(42)
    x = np.array(x0, dtype=float)
    h_x, h_loss = [x.copy()], [loss_fn(x)]
```

```

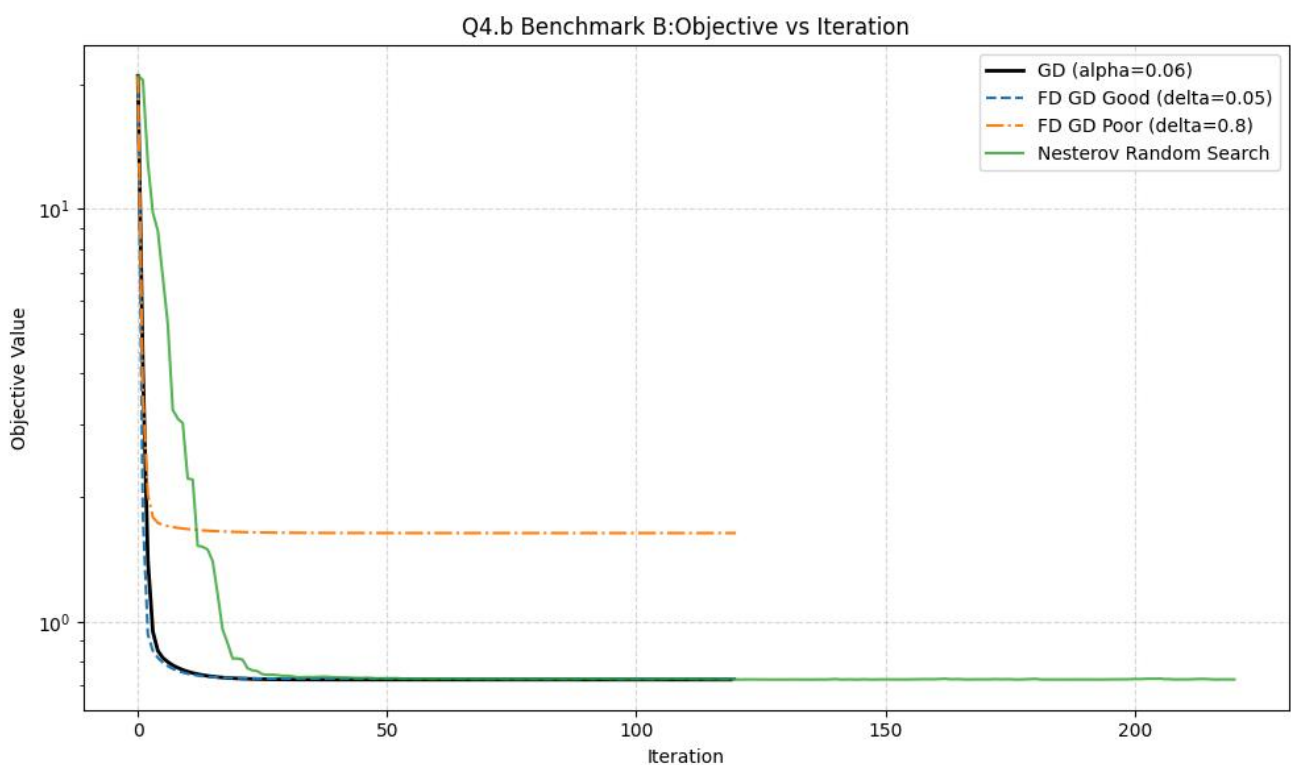
n = len(x)
for _ in range(iters):
    u = np.random.randn(n)
    u = u / np.linalg.norm(u) # Random unit vector

    dir_deriv = (loss_fn(x + delta * u) - loss_fn(x)) / delta
    x = x - alpha * dir_deriv * u

    h_x.append(x.copy())
    h_loss.append(loss_fn(x))
return np.array(h_x), np.array(h_loss)

```

To compare the performance of exact differencing, forward finite differencing with different parameters, and Nesterov stochastic search, I applied the above algorithms to benchmark B and plotted the objective-iteration curves as shown below.



FD GD, $\delta = 0.05$ (blue line): The blue dashed line almost perfectly coincides with the black benchmark line, showing no oscillations. The target value reaches its minimum after approximately 10 iterations, with both following exactly the same trajectory and speed to reach the valley.

This demonstrates that at the $\delta = 0.05$ —a very small scale—the tangent line calculated by forward finite difference has infinitely approximated the true tangent line. It reproduces the true gradient with virtually zero error, exhibiting no difference from GD; here, it is the best-performing algorithm.

FD GD, $\delta = 0.8$ (orange line): After an extremely brief initial descent, the orange line stalled. In the first few iterations, the orange line rapidly descended to a value close to the 10^0 level; although the descent rate was acceptable, it subsequently stalled and failed to reach the minimum.

I believe this is because the $\delta = 0.8$ is too large. In Benchmark B, which features a terrain with slight undulations, a step size of 0.8 causes the algorithm to skip over minor local

depressions, resulting in distorted slope measurements. This false gradient information causes the algorithm to stall at a point that is not the optimal solution.

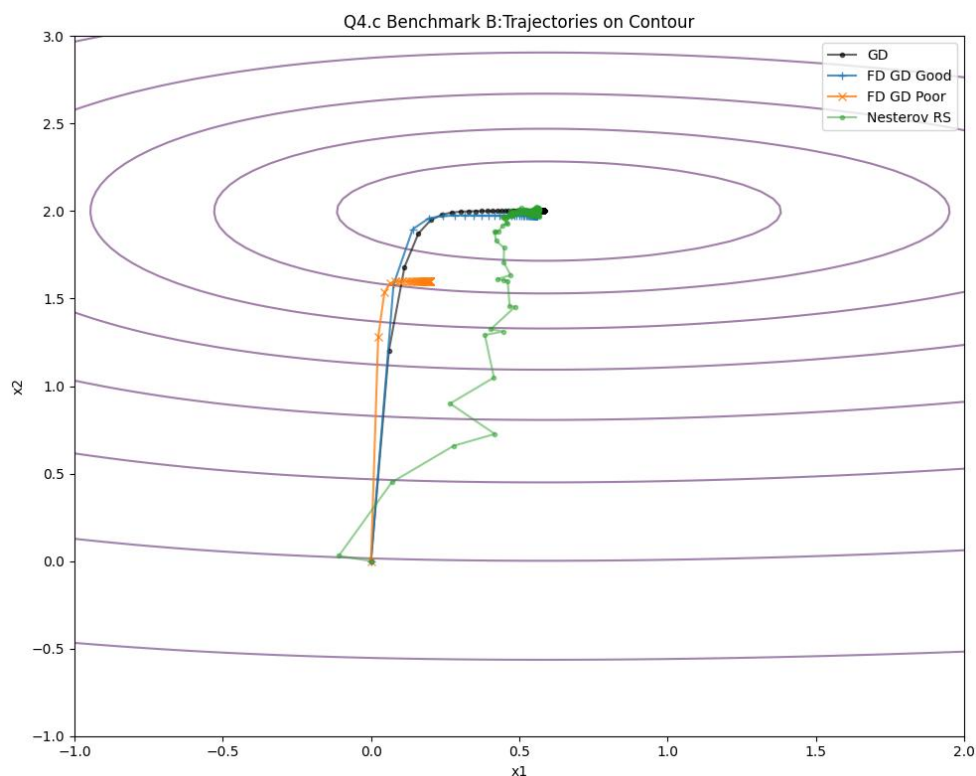
Nesterov Stochastic Search (green line): The descent trajectory of the green line exhibits a distinct stair-step pattern and required more iterations than GD and finite difference methods to eventually reach the local minimum. This stair-step trajectory is due to the fact that the direction of each step (u_k) is randomly determined: when the direction is guessed incorrectly (pointing uphill), the algorithm pauses updates or updates by a minimal amount, resulting in a horizontal plateau on the graph; when the direction is guessed correctly (pointing downhill), the curve undergoes a vertical drop.

Although the convergence curve of the green line is not smooth and the speed is slow, it ultimately succeeds in reaching the minimum. Furthermore, in terms of code efficiency, the forward finite difference algorithm requires $n+1$ function calls per iteration to evaluate all directions in n -dimensional space, whereas random search requires only 2 function calls regardless of the number of dimensions. It incurs lower computational costs and achieves higher efficiency in high-dimensional spaces.

The above experiments demonstrate that the forward finite difference method achieves high accuracy under small parameter conditions and perfectly replicates the descent of the exact gradient. However, it is extremely sensitive to hyperparameters: an excessively large step size can cause truncation errors, leading to premature convergence, while an excessively small step size increases computational overhead. In contrast, although Nesterov stochastic search has large per-step variance and a non-smooth convergence trajectory, it is computationally efficient and ultimately finds the optimal solution.

Q4. C

The contour plots of the optimization trajectories for the three algorithms on benchmark B are shown below.



Exact GD (black line) and FD GD, $\delta = 0.05$ (blue line): In the figure, the blue and black lines almost completely overlap. Both follow an L-shaped trajectory, first rapidly descending

along the steepest x_2 axis to reach the valley, and then approaching the minimum along the gentler x_1 axis. This indicates that when the difference step size is sufficiently small, the finite difference method yields a slope that is indistinguishable from the true slope.

FD GD, $\delta = 0.8$ (orange line): The orange line also descends along the x_2 axis in the initial phase, but suddenly turns at the $x_2 \approx 1.6$ position, after which the points cluster densely together.

This confirms the truncation error mentioned in Q4(b). Because the search step size in $\delta = 0.8$ is too large, when the algorithm extends its search along the x_1 or x_2 axes, it directly skips over subtle terrain curves, leading it to incorrectly conclude that the current location is the valley floor while still halfway up the slope. Ultimately, it becomes stuck due to this false terrain information.

Nesterov Random Search (green line): The green line appears as a zigzag trajectory approaching the minimum.

Because NRS randomly determines its direction in the entire space at every step: when the randomly generated direction happens to align with the normal to the contour line, it takes an effective step; when the random direction is nearly tangent to the contour line or points in the opposite direction, it can only make an extremely small adjustment. Although NRS takes a longer path, its general direction still points toward the center of the minimum. This proves that even if the direction of each step is purely random, as long as corrections are made based on the sign of the directional derivative, the trajectory will still converge.

Q4. D

In this task, I implemented the Nelder-Mead simplex method and included a constant-step gradient descent (GD) as a comparison benchmark.

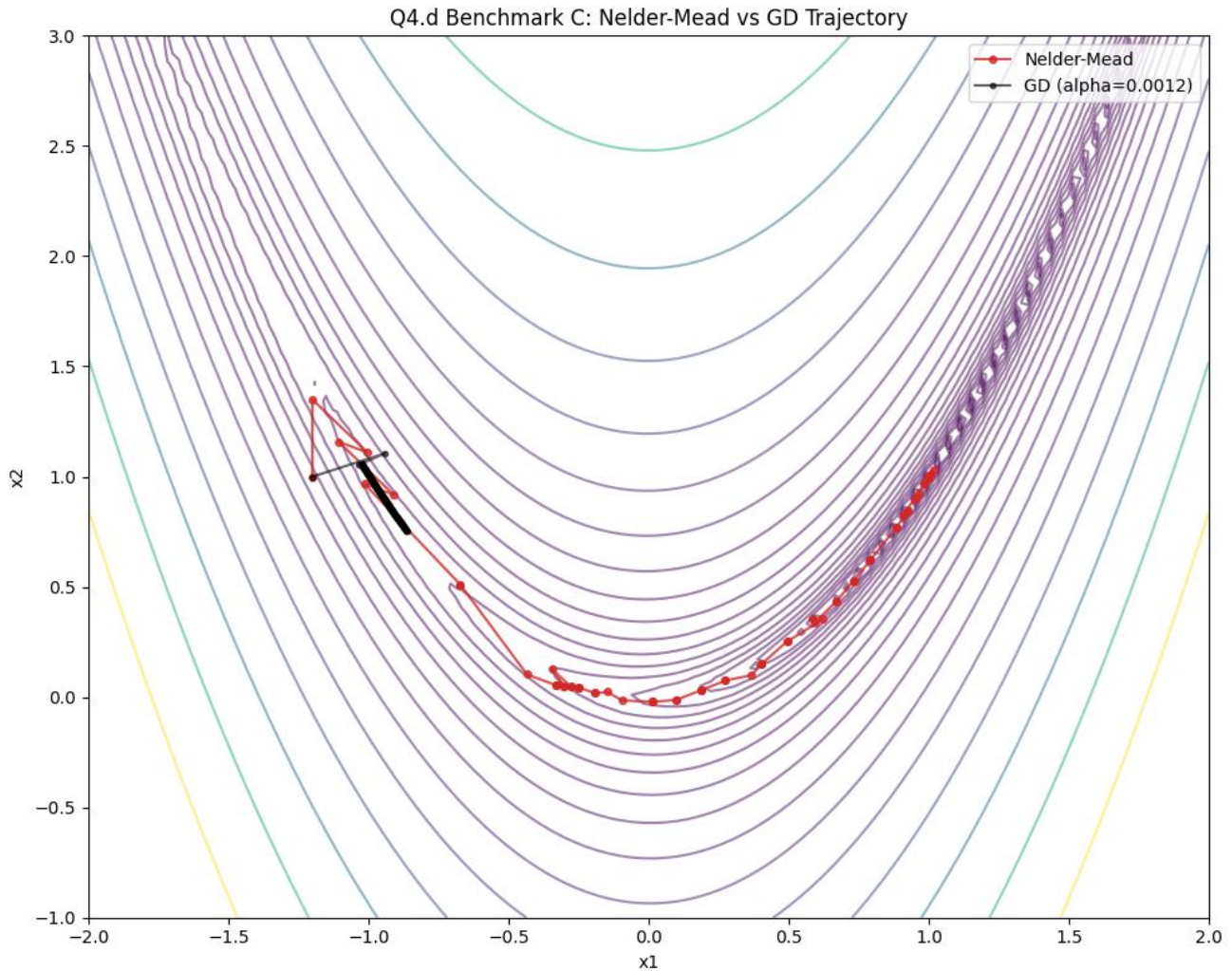
Unlike traditional gradient descent, the Nelder-Mead method maintains a simplex consisting of $n+1$ vertices in n -dimensional space; in Benchmark C, this is a triangle. It calculates the objective function values at all vertices of the simplex, discards the vertex with the highest function value, and generates a new vertex along the direction of the centroid, causing the simplex to move toward the optimal region. If no improvement is possible, the simplex contracts toward the optimal point.

In the code, we use x_0 as the base point and extend it by a step size of 0.35 along the x_1 and x_2 axes, forming an isosceles right triangle. During the underlying iteration of `scipy.optimize`, the algorithm calculates the function values at the three vertices of the triangle and identifies the worst-performing point. It then flips this worst point along the line connecting the other two vertices. If the terrain is better after the flip, it expands further; if the terrain worsens, it contracts. This continues until it contracts to the optimal point.

```
init_simplex = np.array([
    x0_C,
    np.array(x0_C) + np.array([0.35, 0.0]),
    np.array(x0_C) + np.array([0.0, 0.35])
])
res_nm = opt.minimize(loss_C, x0_C, method='Nelder-Mead',
                    options={'initial_simplex': init_simplex, 'maxiter': 160},
                    callback=callback_nm)
```

We apply the implemented Nelder-Mead algorithm to benchmark C and plot the optimization trajectory, as shown in the figure below. For comparison, I have included GD as

a baseline. It should be noted that the trajectory plotted for the Nelder-Mead algorithm represents the centroid of the triangle at each iteration.



The black GD trajectory traced only a very brief zigzag near the starting point ($-1.2, 1.0$), then condensed into a dense cluster of black dots and came to a standstill.

The red trajectory. In the early stages, the red line made several U-turns and jumps because its triangle was constantly flipping to adapt to the steep valley walls; once it entered the flat, curved valley floor, the red line followed the canyon floor all the way to the optimal point.

This is because Nelder-Mead is completely unaffected by gradients; it only compares the relative positions of the triangle's three vertices. As long as it can distinguish which side is higher and which is lower, its triangle will flip toward the lower direction. This demonstrates that Nelder-Mead has a very strong ability to adapt to terrain.

Q4. E

As required by the problem statement, I implemented the grid search optimization algorithm.

Grid search discards all terrain speculation and performs a systematic, exhaustive scan. It does not compute gradients; instead, it selects a number of points along the coordinate axes for calculation, completely unaffected by terrain.

In the code implementation, I first used `np.linspace` to uniformly divide the range $x_1 \in [-2, 2]$ and $x_2 \in [-1, 3]$ into 55 intervals. I then called `np.meshgrid` to form a coordinate grid

from all the intersection points. Using a loop, I iterated through all these coordinate points to evaluate the loss. The algorithm does not update any parameters; it only stores the optimal solution found during the iteration.

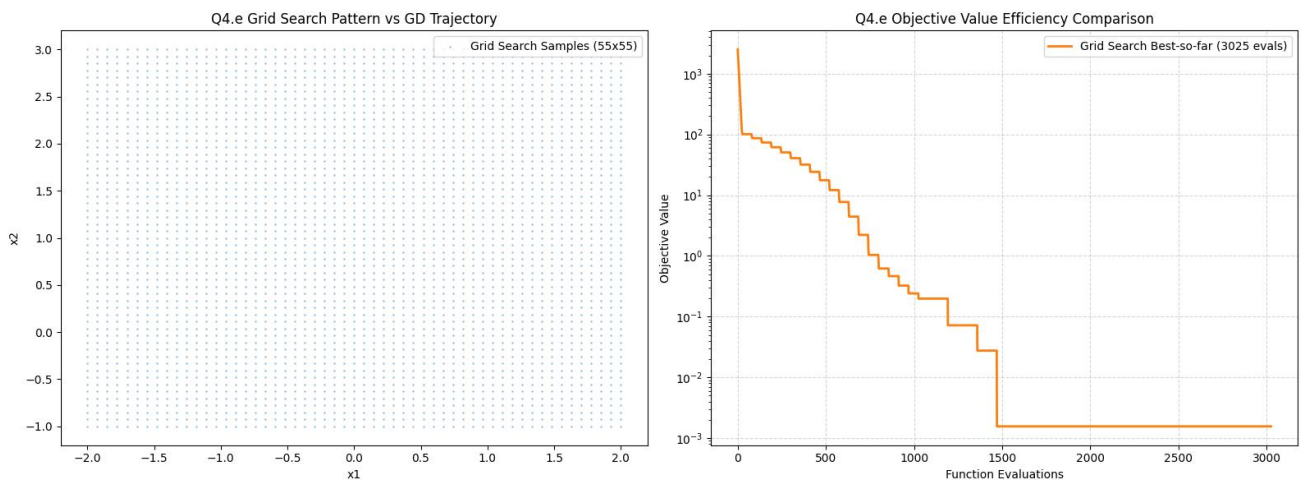
```
def run_grid_search(loss_fn, x1_bounds, x2_bounds, grid_size):
    x1 = np.linspace(x1_bounds[0], x1_bounds[1], grid_size[0])
    x2 = np.linspace(x2_bounds[0], x2_bounds[1], grid_size[1])
    X1, X2 = np.meshgrid(x1, x2)

    best_val = np.inf
    h_best_val = []

    for i in range(grid_size[0]):
        for j in range(grid_size[1]):
            pt = np.array([X1[i, j], X2[i, j]])
            val = loss_fn(pt)
            if val < best_val:
                best_val = val
            h_best_val.append(best_val)

    return X1, X2, np.array(h_best_val)
```

The following shows the sampling points and optimal values from the benchmark C-grid search.



Left figure: In the scatter plot on the left, 3,025 sampling points are evenly distributed. However, in the Rosenbrock valley, the only truly valuable region is the valley floor, yet the grid search performs a great deal of useless exploration, which is a significant waste of resources.

Right figure: The descent curve shows a step-like decline. The x-axis indicates that the algorithm performed approximately 1,500 function evaluations, ultimately reducing the function loss to the order of 10^{-3} . Recall the Newton's method tested in Q3, which achieved this precision in fewer than 5 iterations. Furthermore, after approximately 1,500 evaluations, the orange line becomes a horizontal line. This is not because it has converged to a local minimum, but because we only divided the space into a 55x55 grid, and none of the 3,025 points in this grid happened to fall exactly on the true local minimum at (1, 1). Therefore, no matter how many evaluations are performed, if the grid is not refined, the value will remain at this point.

Based on the above experiment, it can be concluded that while grid search is completely unaffected by terrain, it is essentially an extremely inefficient brute-force method. Its optimization accuracy is strictly limited by the grid resolution, and it wastes a significant amount of computational power. Therefore, grid search is only suitable for rough searches in low-dimensional spaces. In practical high-precision or high-dimensional optimization tasks, it is not only extremely costly but also unable to provide an exact solution.

Q4. F

Based on the performance of the four gradient-free algorithms in the above experiments, the following table summarizes their results.

Algorithm	Accuracy	Computational Cost	Robustness
Finite Difference	Highly parameter-dependent: Perfect parameters can replicate GD, but improper parameters will result in truncation errors	High: Each update requires $n+1$ function calls to compute each dimension	Very low: If parameters are set improperly, the algorithm can be easily misled by minor local irregularities and fall into an infinite loop
Random Search	Medium: Trajectory exhibits stair-step jaggedness; large variance per step	Low: Regardless of the number of dimensions, only 2 function calls are required per step	Medium: Takes a longer path but does not stall; the expected trajectory still points toward the center of the minimum
Nelder-Mead	High: Achieves accuracy comparable to the analytical gradient by continuously contracting simplices	Medium: Maintains $n+1$ vertices, continuously evaluating and comparing	Extremely Strong: Does not rely on gradients; compares only high and low values, enabling adaptation to extreme terrain
Grid Search	Very Low: Limited by grid resolution; unable to find true extrema	Extremely High: Requires traversing every point	Extremely strong: Brute-force search completely ignores terrain

Accuracy: Nelder-Mead performs best. By continuously contracting the simplex at the valley floor, it achieves extremely high physical accuracy comparable to that of the analytical gradient. The accuracy of forward finite differences depends entirely on the exploration step size. An appropriate step size can reproduce the exact gradient, but at a high computational cost. Nesterov stochastic search has moderate accuracy; limited by the variance of random sampling, its approach to the minimum involves some error. Grid search has the lowest accuracy, as its final solution depends entirely on the grid resolution.

Cost: Nesterov stochastic search has the lowest computational cost. Regardless of the number of dimensions in the parameter space, each forward step requires only two function calls. Nelder-Mead has moderate computational cost, requiring maintenance of function values at only a small number of vertices. Forward finite difference has high computational

cost, requiring $n+1$ function calls per update, with cost increasing with dimensionality. Grid search has extremely high computational cost, with consumption increasing as grid resolution increases.

Stability: Nelder-Mead exhibits exceptional stability because it does not compute the absolute gradient; instead, it compares the relative heights of vertices, allowing it to adapt to flat and curved pathological valleys in the Rosenbrock function. Nesterov stochastic search is relatively stable; although the trajectory appears jagged, it generally converges toward the optimal solution. Forward finite difference has very poor stability and is highly dependent on the exploration step size, which can easily lead to stagnation. Grid search is the most stable; it is completely unaffected by terrain and performs a brute-force search.

Q5

Q5.A

For this task, in accordance with the problem statement, I implemented three optimization algorithms: unconstrained GD, projected GD, and penalized GD.

Below are the implementations of each algorithm:

1. Projected Gradient Descent

Projected gradient descent allows parameters to move in the normal gradient direction at each update step; however, if the updated function crosses the feasible region (i.e., $x_1 < 0.5$), it uses geometric projection to pull the parameters back to the boundary of the feasible region.

For the implementation of the projected gradient descent algorithm, I first perform gradient descent using the basic gradient update formula. After calculating the updated point, I project it onto the feasible region. The corresponding projection formula is: $x_1 \leftarrow \max(0.5, x_1)$

```
def run_projected_gd(x0, alpha, iters):
    x = np.array(x0, dtype=float)
    h_x, h_loss, h_viol = [x.copy()], [loss_B(x)], [get_violation(x)]
    for _ in range(iters):
        x = x - alpha * grad_B(x)
        x[0] = max(0.5, x[0])
        h_x.append(x.copy())
        h_loss.append(loss_B(x))
        h_viol.append(get_violation(x))
    return np.array(h_x), np.array(h_loss), np.array(h_viol)
```

2. Penalty Method

The penalty method differs from projected gradient descent, which forcibly pulls the function back from out-of-bounds regions. Instead, the penalty method modifies the original objective function by imposing a penalty when the function goes out of bounds, thereby guiding it back into the feasible region.

In practice, when the function goes out of bounds, a force in the opposite direction is applied to the gradient during the update to push it back into the feasible region.

In each update, if the function has not gone out of bounds, the original function gradient is used; if it has gone out of bounds, a penalty term is added to the gradient. The new objective function with the penalty term is as follows:

$$F_{\lambda}(x) = f(x) + \lambda \max(0, 0.5 - x_1)$$

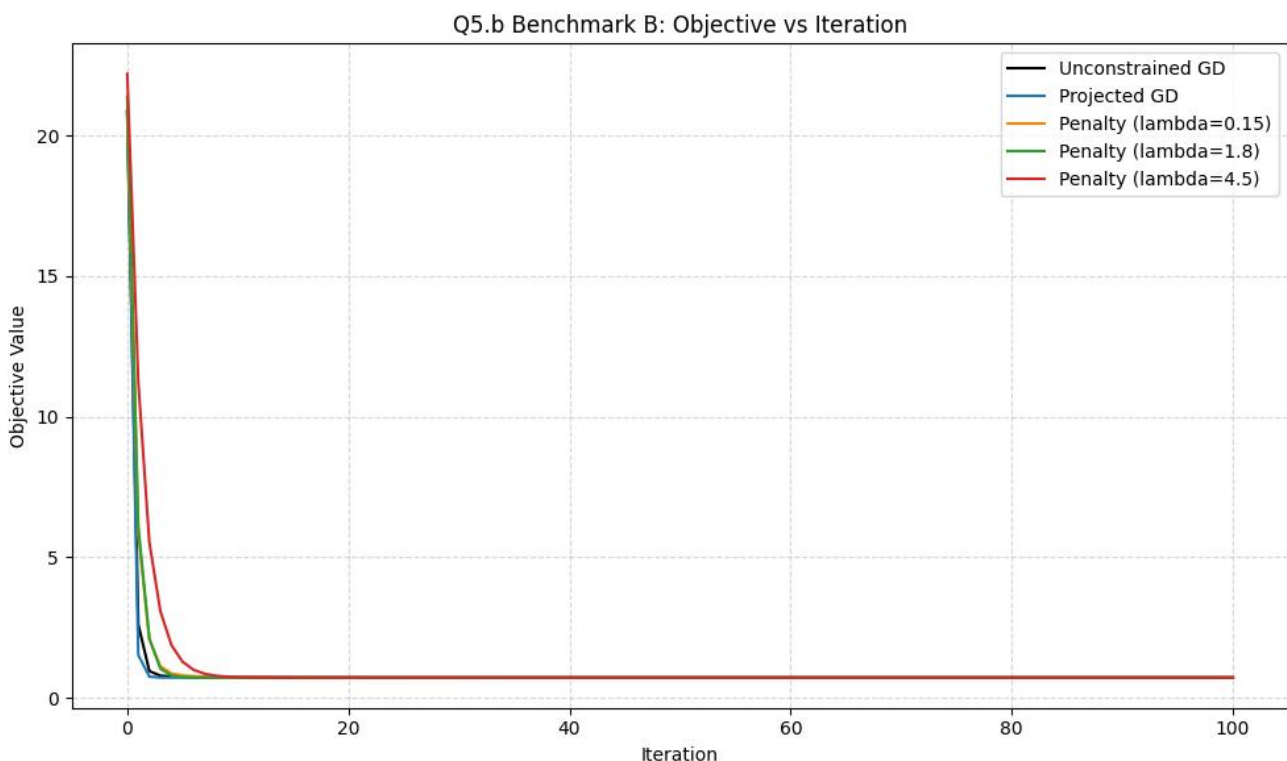
```
def penalty_obj(x, lam):
    return loss_B(x) + lam * max(0, 0.5 - x[0])

def penalty_grad(x, lam):
    g = grad_B(x)
    # Add penalty gradient if in infeasible region
    if x[0] < 0.5:
        g[0] -= lam
    return g

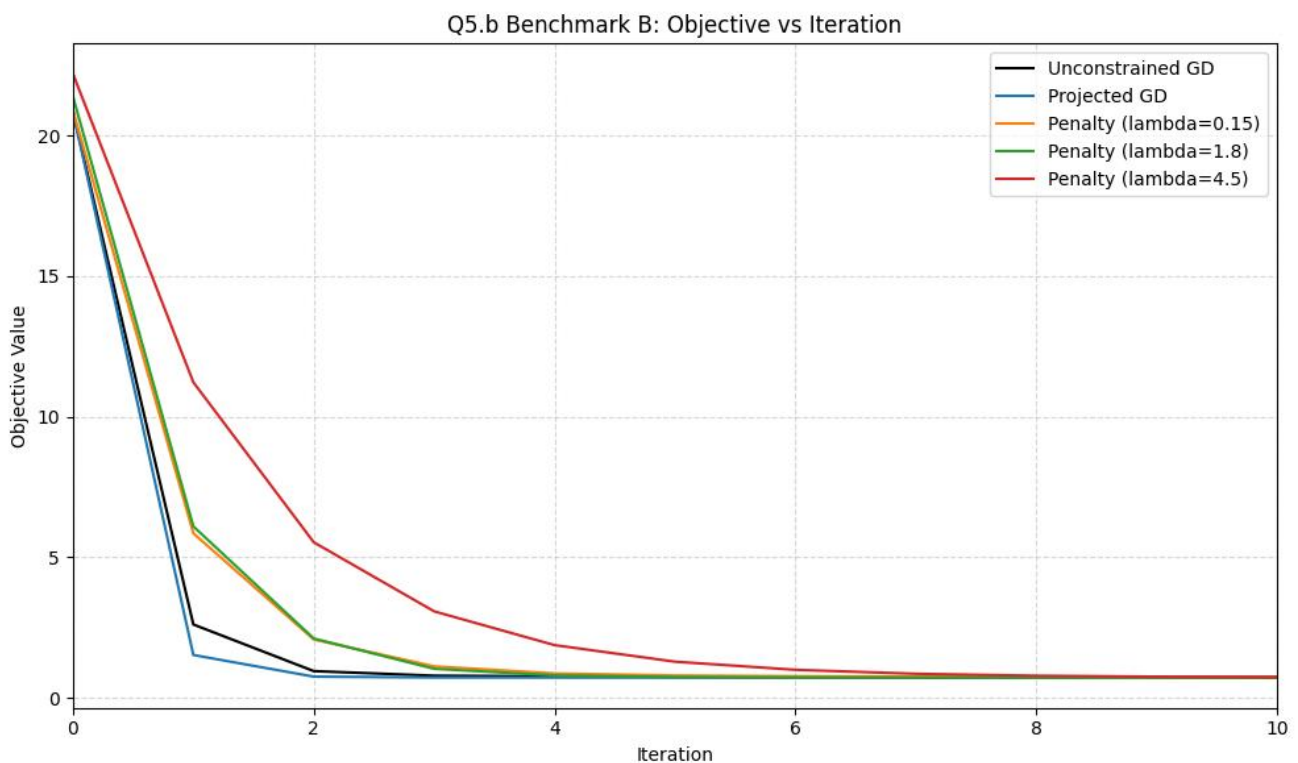
def run_penalty_gd(x0, alpha, lam, iters):
    x = np.array(x0, dtype=float)
    h_x, h_obj, h_viol = [x.copy()], [penalty_obj(x, lam)], [get_violation(x)]
    for _ in range(iters):
        x = x - alpha * penalty_grad(x, lam)
        h_x.append(x.copy())
        h_obj.append(penalty_obj(x, lam))
        h_viol.append(get_violation(x))
    return np.array(h_x), np.array(h_obj), np.array(h_viol)
```

Q5. B

To compare the performance of penalty methods and projected gradient descent with different parameters, I applied the above algorithm to benchmark B and plotted the objective-iteration curve as shown below.



As you can see, there is little difference in the algorithms' performance. To make it easier to observe, I have limited the horizontal axis of the graph so that the subtle differences between the algorithms are more apparent.



Black line (GD): The black line is completely unaffected by the feasible region; it reaches the minimum after just ten iterations starting from the initial point. Its convergence speed is second only to Projected GD.

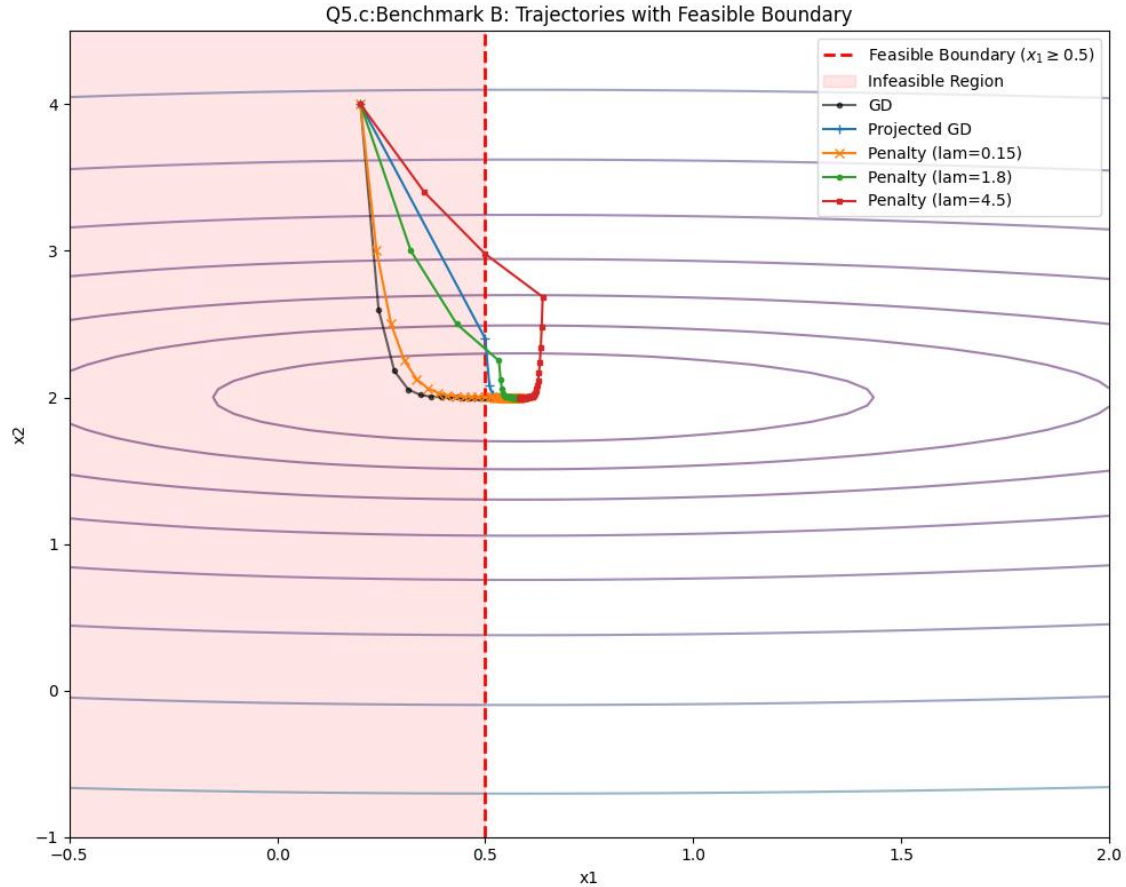
Blue line (Projected GD): As can be seen, the blue line drops vertically in the first step. This indicates that the projection mechanism was triggered. Since the initial point (0.2, 4.0) is outside the feasible region, it was projected onto the boundary of the feasible region in the first step. This brought it very close to the optimal point within the feasible region, resulting in a significant drop in the function value. Subsequently, the blue line begins to slowly approach the minimum. Visually, the projection method appears to be the fastest algorithm, reaching the optimal solution within the first few steps. I believe this is primarily due to the projection being triggered in the first step, which directly accelerates the algorithm's convergence.

Orange line (Penalty $\lambda = 0.15$), Green line (Penalty $\lambda = 1.8$), Red line (Penalty $\lambda = 4.5$): Zooming in on the image reveals that the initial starting heights of the three lines increase progressively. The orange line starts at the lowest point, the green line in the middle, and the red line at the highest initial target value. This is because the initial point (0.2, 4.0) is not within the feasible region, so the algorithm is immediately subject to a penalty of $0.3 \times \lambda$ from the start. The larger the penalty factor, the heavier the penalty imposed at the beginning.

In the early stages of iteration, all three curves drop rapidly and hit bottom within the first 10 iterations or so. More importantly, they eventually converge completely at the bottom of the graph, reaching the same optimal value as unconstrained GD. The difference lies in the slight variation in convergence speed; algorithms with larger penalty parameters converge faster.

Q5. C

The contour plots of the optimization trajectories for the three algorithms on benchmark B are shown below.



Black line (Unconstrained GD): Starting from the origin, the black line is completely unaffected by the infeasible region. It rapidly descends along the X_2 axis to the valley floor and then slowly approaches the optimal point along the X_1 axis.

Blue Line (Projected GD): After starting, the blue line immediately jumps diagonally downward to the boundary of the feasible region. Subsequently, it exhibits behavior similar to standard GD, first moving rapidly along the X_2 axis and then slowly approaching the optimal point along the X_1 axis.

This demonstrates the clipping mechanism of Projected GD. Whenever the algorithm crosses the boundary, the projection operation forces its x_1 coordinates to reset to 0.5. As can be seen, while the trajectories of the other algorithms require several steps to enter the feasible region, Projected GD requires only one step, making it highly efficient.

Orange line (Penalty $\lambda = 0.15$), green line (Penalty $\lambda = 1.8$), red line (Penalty $\lambda = 4.5$): These three lines use the same algorithm. However, their trajectory behaviors differ. Their motion trajectories are governed by the superposition of two forces: the gradient of the objective function and the thrust exerted by the penalty function, which forces them toward the right-hand feasible region. The magnitude of the penalty factor (λ) directly determines the magnitude of this thrust.

Orange line (weak penalty, $\lambda = 0.15$): Due to the low penalty strength, its overall trajectory is almost identical to that of GD, with only a slight shift to the right.

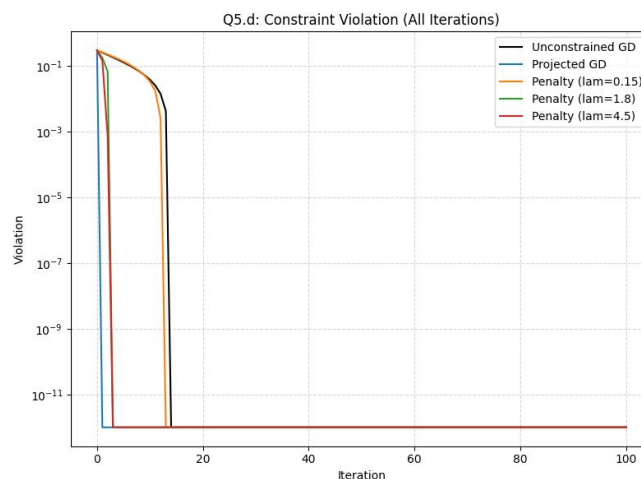
Green line (moderate penalty, $\lambda = 1.8$): The green line experiences a noticeable push to the right; the function trajectory enters the feasible region in just 3 steps, moving diagonally downward to the right, and then slowly approaches the minimum.

Red line (Strong penalty, $\lambda = 4.5$): Under high penalty, the force pushing the function away from the infeasible region to the right is much stronger, allowing it to escape the infeasible region in just two steps. After entering the feasible region, the rightward penalty force disappears, and the function is then guided entirely by the downward gradient, moving almost vertically downward toward the optimal point.

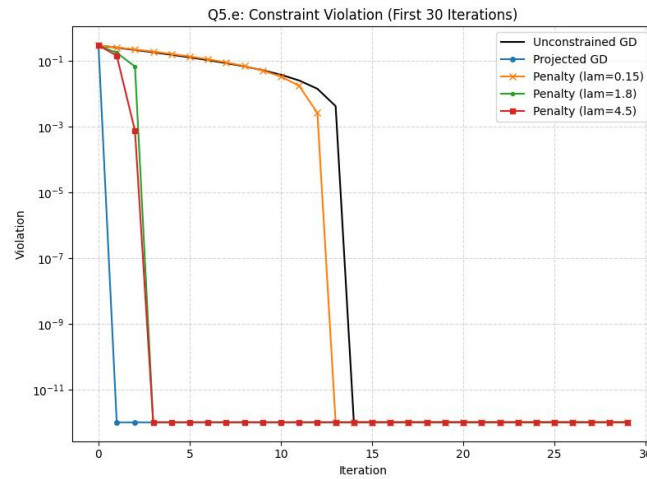
Experiments have shown that when solving constrained optimization problems, projection is highly efficient at forcing parameters to return to and conform to the feasible region in a single step. In contrast, with the penalty method, the convergence rate depends entirely on the magnitude of the penalty term: the heavier the penalty, the more quickly the algorithm is forced to escape the infeasible region.

Q5 D, E

In these two tasks, to compare the efficiency of each algorithm in enforcing feasibility, I calculated the constraint violation score $\max(0, 0.5 - x_1)$ at each step and visualized the results.



From the global perspective of Q5.d (steps 0–100), it can be seen that all algorithms achieved zero violation degree within the first 15 steps, meaning they escaped the infeasible region. Among them, Projected GD follows an almost vertical line, achieving zero violation degree in the very first step. In the penalty method, the speed at which the violation degree reaches zero is related to the penalty level. The high-penalty level (red line) reaches zero the fastest among the penalty methods, but is slightly slower than the projection method. It is worth noting that the violation degree decline curve for the medium-penalty level is almost identical to that of the high-penalty level. To observe this more closely, we limited the number of iterations to 30 to zoom in on the image.



In the zoomed-in view, we can precisely see at which step the algorithm escaped the infeasible region.

Projected GD (blue line): Number of steps to bottom: Step 1. The blue line drops at a right angle to the bottom the moment it starts. This is primarily due to the projection operation; the projection in the first step forces the x_1 coordinates to re-align with the feasible boundary.

Strong penalty ($\lambda = 4.5$, red line): Reaches the bottom in 3 steps. The extremely high penalty creates a massive repulsive force to the right, forcibly pulling the parameters out of the forbidden region.

Moderate penalty ($\lambda = 1.8$, green line): Also reached the bottom in 3 steps. However, it can be seen that in the second step, the violation level under the strong penalty is still lower than that under the moderate penalty.

Weak penalty ($\lambda = 0.15$, orange line): It took approximately 13 steps to reach the bottom, which is almost identical to GD, indicating that the optimization effect on the algorithm is not significant.

Q5. F

Based on the results of the above experiments, I have summarized the absolute speed at which each algorithm enters the feasible region in the table below.

Algorithm	Parameter Settings	Time to Find the Minimum	Constraint Enforcement Mechanism and Performance
Projected GD	N/A	1 Step	Hard Constraint: Most efficient. Forces parameters to conform to the boundary in a single step.
Penalty Method	$\lambda = 4.5$ (strong penalty)	3-step	Soft Constraints (High Penalty): Fast convergence. A strong repulsive force to the right forces the parameters to escape the infeasible region as quickly as possible.
Penalty Method	$\lambda = 1.8$ (moderate penalty)	3 steps	Soft Constraints (Moderate Penalty): Fast.

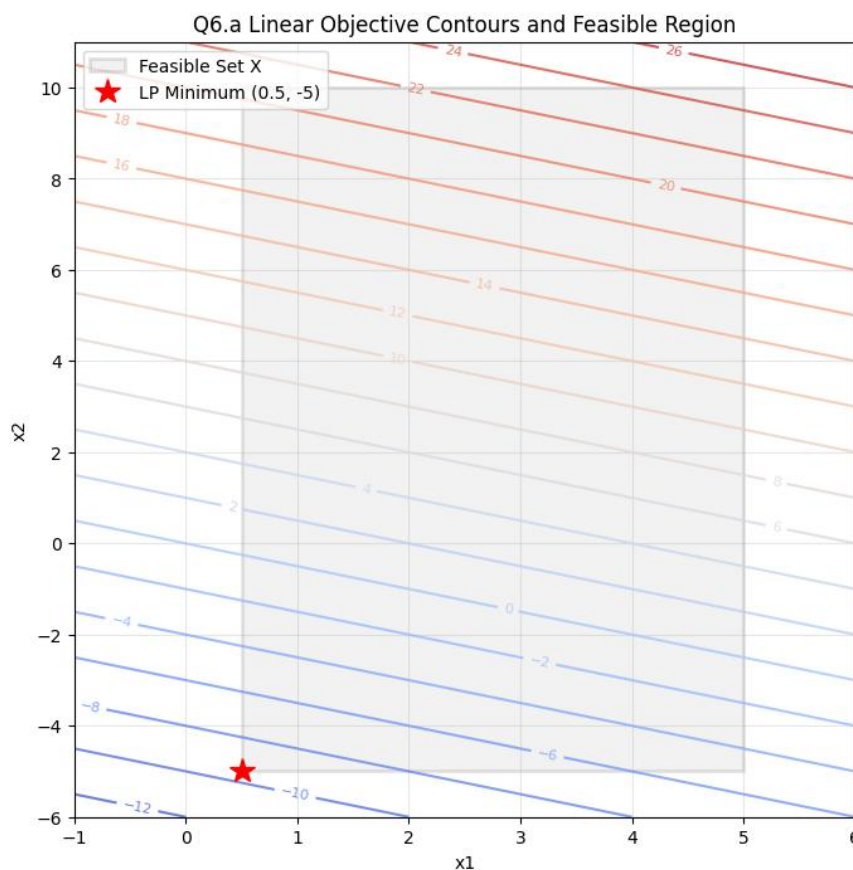
Algorithm	Parameter Settings	Time to Find the Minimum	Constraint Enforcement Mechanism and Performance
Penalty Method	$\lambda = 0.15$ (weak penalty)	13 steps	Soft Constraints: Extremely low efficiency. Virtually indistinguishable from GD without any optimization
Unconstrained	N/A	14 steps	Unconstrained

Projected GD is the fastest method for forcing parameters to satisfy constraint conditions. Regardless of the terrain or how far from the feasible region, it takes only 1 step to enter the feasible region, making it the best solution for this type of problem. In contrast, the escape speed of the Penalty Method is strictly controlled by the magnitude of the penalty factor: increasing the penalty factor can significantly accelerate the entry into the feasible region; whereas Unconstrained GD has no boundary control capabilities whatsoever and is completely unaffected by the feasible region.

Q6

Q6.A

In this task, we defined the constrained feasible region $\mathcal{X} = \{x \in \mathbb{R}^2: 0.5 \leq x_1 \leq 5, -5 \leq x_2 \leq 10\}$ and plotted the contour map of the linear objective function $f(x) = x_1 + 2x_2$, as shown below.



As can be seen from the generated image, the feasible region $\mathcal{X}$ forms a regular rectangle. The contour lines of the objective function $f(x) = x_1 + 2x_2$

consists of a set of parallel lines. Since the gradient of this function is the constant vector $\nabla f(x) = [1, 2]^T$, pointing upward and to the right, the value of the objective function decreases monotonically along the negative gradient direction (i.e., toward the lower-left).

This means that to minimize the objective function, we need to move as far to the lower-left as possible within the feasible region. Under the constraints of the rectangle, the farthest lower-left point we can reach is the lower-left vertex of the rectangle $(0.5, -5)$, and the minimum objective value at this point is -9.5 .

Q6. B

In this task, we investigate the behavior of the Frank-Wolfe algorithm when the minimum of the objective function lies inside the feasible region.

First, regarding the implementation of the Frank-Wolfe algorithm, unlike standard projected gradient descent, the Frank-Wolfe algorithm performs updates by solving a linear subproblem. Its core mathematical update formula consists of two steps:

Finding the optimal vertex: $z_k = \arg \min_{x \in \mathcal{X}} \nabla f(x_k)^T x$

Update: $x_{k+1} = \beta x_k + (1 - \beta) z_k$

Since our feasible region is a standard rectangular box, solving the linear subproblem is very straightforward. We simply check the sign of the gradient: if the gradient is greater than 0 in a given dimension, we take the lower bound for that dimension; otherwise, we take the upper bound. Corresponding code:

```
def solve_lp(grad):
    z1 = X_BOUNDS[0] if grad[0] >= 0 else X_BOUNDS[1]
    z2 = Y_BOUNDS[0] if grad[1] >= 0 else Y_BOUNDS[1]
    return np.array([z1, z2])
```

Subsequently, the algorithm uses the weight β to draw a line segment between the current position and the target vertex, and takes a step of $(1 - \beta)$ forward. This ensures that x_{k+1} will never jump out of the feasible region and will continuously converge toward the optimal solution.

```
def run_frank_wolfe(loss_fn, grad_fn, x0, beta, iters):
    x = np.array(x0, dtype=float)
    h_x, h_z, h_loss = [x.copy()], [], [loss_fn(x)]

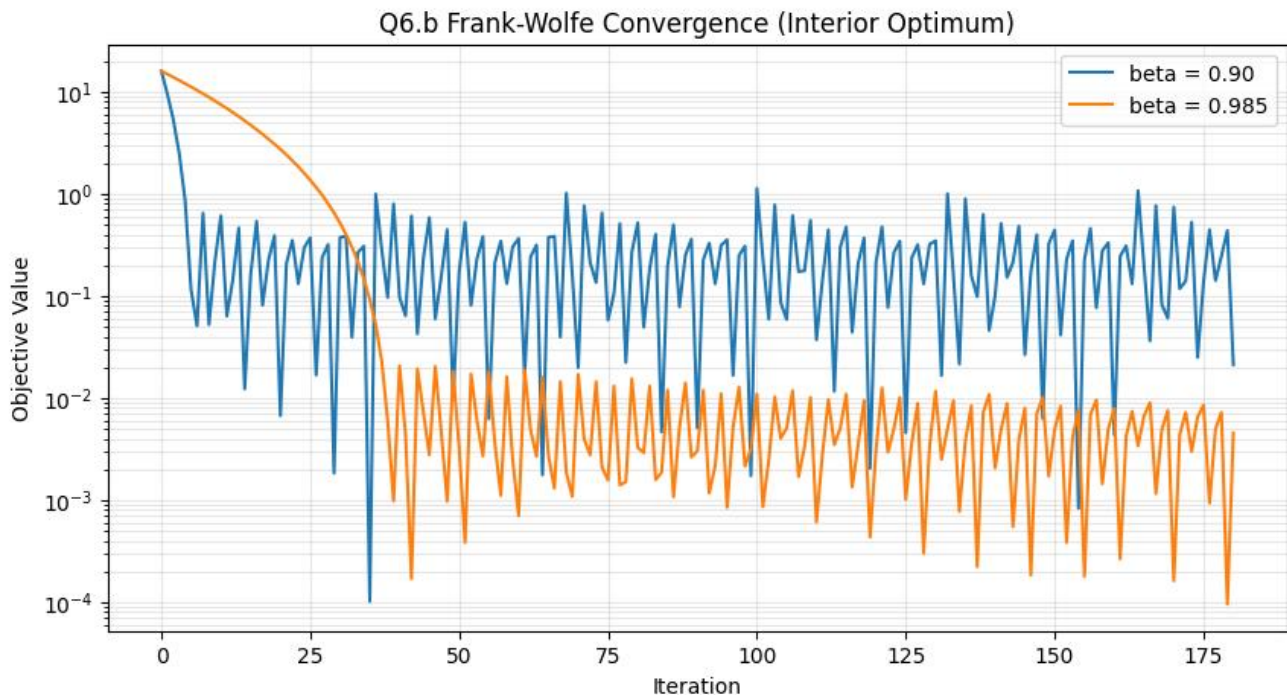
    for k in range(iters):
        # Compute the current gradient
        g = grad_fn(x)
        # Find the optimal corner point z
        z = solve_lp(g)
        h_z.append(z.copy())
        # Update x via convex combination
        x = beta * x + (1 - beta) * z
        h_x.append(x.copy())
```



```
h_loss.append(loss_fn(x))
```

```
return np.array(h_x), np.array(h_z), np.array(h_loss)
```

Apply the above algorithm to the objective function: $f(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 5)^2$, and plot the convergence curve as shown below.



At this point, the global minimum of the objective function is located at $(1, 5)$, which lies within the feasible region.

It can be seen that both curves with different weights of β exhibit severe sawtooth oscillations. I believe this is because the Frank-Wolfe algorithm can only point toward the vertices of the feasible region at each step. When the optimal solution lies within the feasible region, the algorithm cannot move directly toward the optimal point but must oscillate between the vertices surrounding it. As it approaches the optimal point, the pull toward a particular vertex in the next step causes it to deviate from the optimal point, resulting in the sharp oscillations visible in the graph.

Blue line ($\beta = 0.90$): During the first 5 iterations, the function value rapidly decreases to the order of 10^{-1} , after which the convergence curve begins to oscillate violently at the order of 10^{-0} to 10^{-4} . Since $\beta = 0.90$, this implies that the step size taken by the algorithm toward each corner point is relatively large at 0.1, causing the algorithm to overshoot near the optimal solution and remain unable to approach it.

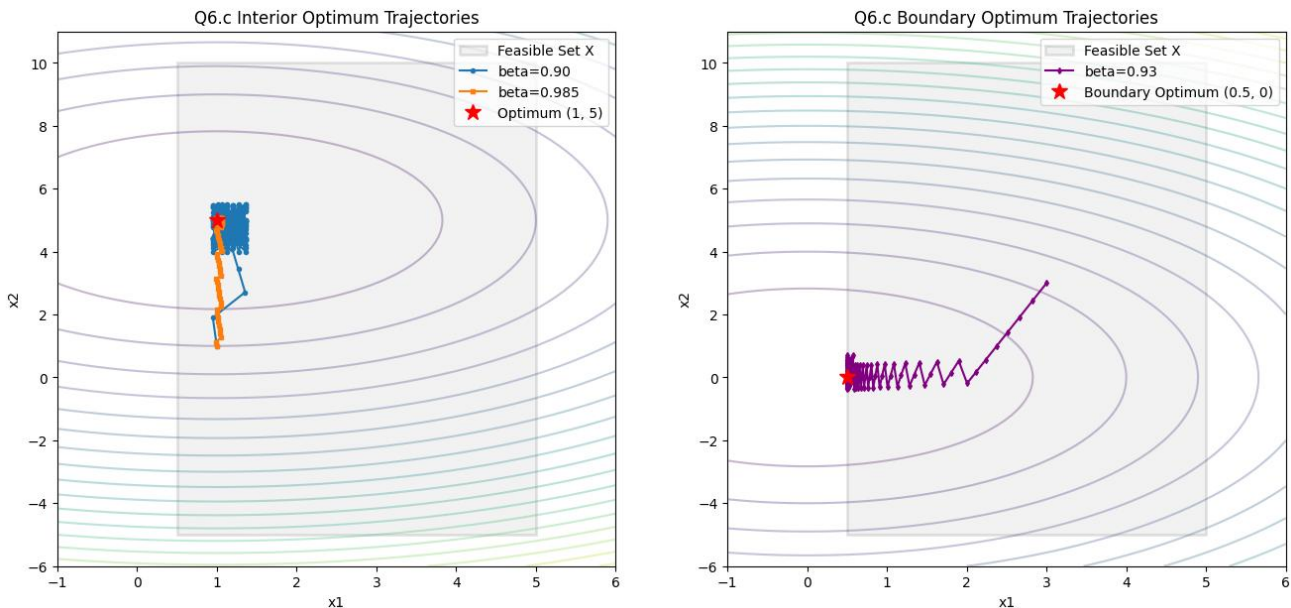
Orange line ($\beta = 0.985$): When we increased β to 0.985, the step size toward the corner point sharply decreased to 0.015. The orange line initially exhibited a very smooth downward trajectory because its steps were small enough to allow it to approach the local optimum closely. Although the orange line still exhibited oscillations in the later stages (after approximately 40 steps) due to the pull of the corner points, the amplitude of these oscillations remained at a low level—between 10^{-3} and 10^{-2} —and its convergence accuracy was higher than that of the blue line.

Experiments show that when the optimal solution lies within the feasible region, the Frank-Wolfe algorithm encounters the pull of corner points, inevitably leading to oscillations. In this case, if β is too small, although a decent convergence rate can be

achieved, severe overshooting occurs, resulting in very poor convergence accuracy. Increasing β reduces the step size toward the corner points, effectively suppressing oscillations and improving approximation accuracy, but at the cost of some convergence speed.

Q6. C

In this task, I applied the Frank-Wolfe algorithm I implemented to two objective functions: $f(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 5)^2$ and $f(x_1, x_2) = x_1^2 + x_2^2$. The optimal solution for the former lies within the feasible region, while the optimal solution for the latter lies on the boundary of the feasible region. Below are contour plots of the Frank-Wolfe optimization trajectories for both cases.



1. Left Plot: Interior Optimum

Setting: The true unconstrained optimum is located at $(1, 5)$, inside the rectangular feasible region.

Blue Line ($\beta=0.90$): In the early stages, the blue line approaches the optimum in just a few steps, with the trajectory appearing jagged. Due to the low weight assigned to historical positions, the algorithm takes relatively large steps toward the corners. This causes significant overshoot near the optimum, resulting in the algorithm oscillating back and forth without ever reaching the optimum.

Orange line ($\beta=0.985$): Similarly, in the early stages, the overall trajectory appears jagged, but the amplitude is significantly smaller than that of the blue line. However, it takes more iterations to approach the optimal point. This is because the high historical weight limits the distance it is pulled toward the corners at each step. Consequently, its trajectory appears more restrained, with oscillations confined near the optimal point $(1, 5)$, demonstrating superior convergence accuracy.

2. Right Plot: Boundary Optimum

Setting: The true optimum of the objective function is at the origin $(0, 0)$, but due to the hard constraint $x_1 \geq 0.5$, the constrained valid optimum is located at $(0.5, 0)$ on the boundary.

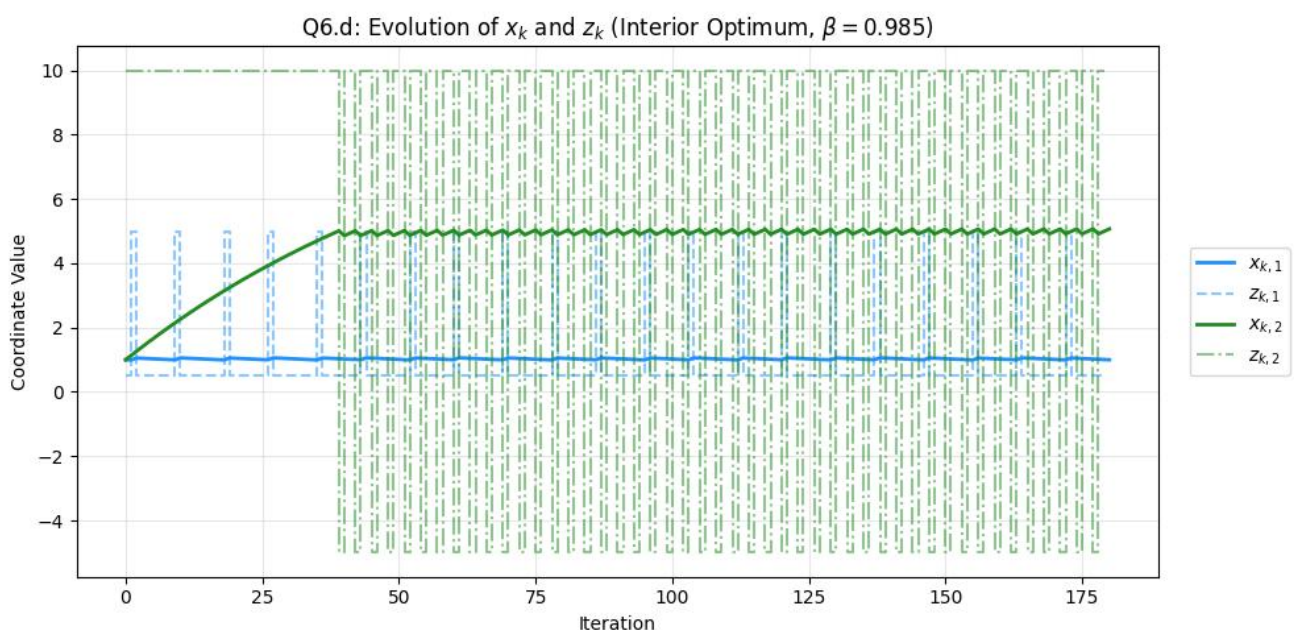
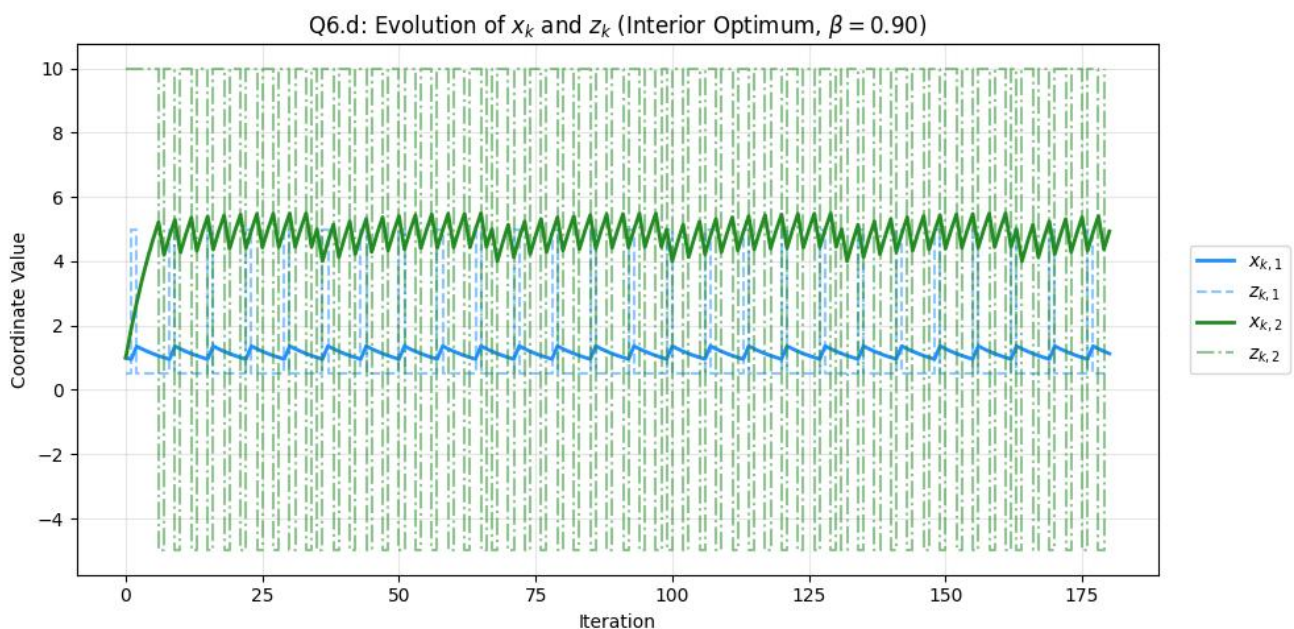
Purple line ($\beta=0.93$): Starting from $(3, 3)$, it is initially guided by the bottom-left corner and moves in a straight line downward and to the left for a few steps. At $x_1=2$, the purple line's trajectory begins to form an alternating "Z"-shaped zigzag pattern. It gradually approaches

the left boundary and, upon reaching the boundary at $x_1=0.5$, ceases to move forward, instead oscillating repeatedly up and down. This is because, during the initial stage, since the parameters x_1, x_2 are all greater than 0, both components of the gradient are positive. Therefore, the optimal corner locked by the algorithm is always the bottom-left corner of the feasible region. When the algorithm crosses the boundary of $x_2 = 0$ due to step size inertia, x_2 becomes negative, the sign of the vertical gradient reverses, and the target corner switches to the top-left corner. Subsequently, as the algorithm is pulled toward the top-left corner, it crosses the midline again and switches back to the bottom-left corner. This process repeats.

Q6. D

This section analyzes the driving mechanisms of the Frank-Wolfe algorithm in different environments by observing the evolution of the current position coordinates x_k and the target vertex coordinates z_k obtained from the linear subproblem with respect to the number of iterations.

Internal optimal solution

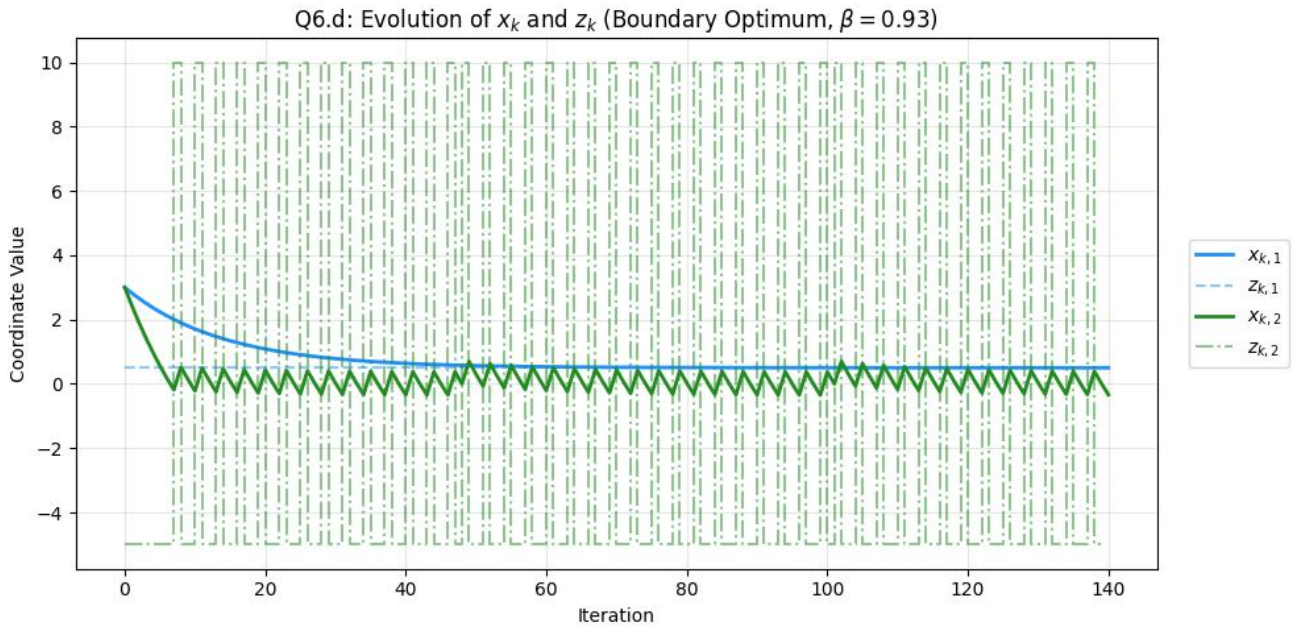


z_k : Regardless of the value of β , the two dashed lines in the figure (the light blue $z_{k,1}$ and the light green $z_{k,2}$) exhibit high-frequency, square-wave-like jumps. $z_{k,1}$ switches between 0.5 and 5.0, while $z_{k,2}$ switches between -5.0 and 10.0. This demonstrates that when the algorithm oscillates near the local minima (1, 5), both components of the gradient continuously change signs, causing the linear subproblem to frequently switch targets between the four corners of the rectangle.

Figure1($\beta = 0.90$) : Due to the low weight of historical inertia, the algorithm is extremely susceptible to jumps to corner points. The green $x_{k,2}$ is pulled significantly between 4 and 6, and the blue $x_{k,1}$ also exhibits visually noticeable, sharp sawtooth waves around 1.0.

Figure2($\beta = 0.985$) : Extremely high β values act as a damping mechanism. Although the underlying target vertices still jump across the entire screen, the solid lines $x_{k,1}$ and $x_{k,2}$ have largely stabilized along the 1.0 and 5.0 reference lines. By relying on extremely small step sizes, the algorithm reduces oscillations and achieves extremely high convergence accuracy.

Evolution of the Boundary Optimal Solution



The third figure ($\beta = 0.93$, boundary optimal) shows that the objective function reaches its minimum at the origin (0, 0), while the valid optimal solution under constraints lies on the boundary at (0.5, 0).

The light blue dashed line ($z_{k,1}$) remains at 0.5 throughout the entire process, appearing as a horizontal straight line with no changes. Since x_1 is always greater than or equal to 0.5, the horizontal gradient is always directed to the left, and the objective is always the left boundary. Guided by this, the dark blue line $x_{k,1}$ exhibits an extremely smooth decay curve, gliding toward 0.5 without any oscillations.

The light green dashed line ($z_{k,2}$) remains locked at the lower bound of -5.0 for the first 8 steps or so. At this point, both the horizontal and vertical targets remain constant, corresponding to the initial linear movement in the trajectory plot. When the dark green line ($x_{k,2}$) drops below the 0-axis, the light green dashed line ($z_{k,2}$) begins to oscillate in a square wave pattern between -5.0 and 10.0. This indicates that after the algorithm crosses the midline, the sign of the vertical gradient reverses, causing the linear subproblem to alternate

between the bottom-left and top-left corners. Consequently, the dark green $x_{k,2}$ is pulled into a sawtooth wave oscillating around the 0-axis.

Q6. E

Through an analysis of the optimization trajectory, convergence curve, and coordinate system evolution, I have summarized the similarities and differences in the behavior of the Frank-Wolfe (FW) algorithm when handling problems with interior and boundary optimums.

1. Interior Optimum

When the true minimum lies inside the feasible region, the components of the gradient frequently change sign as the algorithm overshoots its position while approaching the minimum. This causes the target vertex (z_k) to jump continuously between the four corners of the rectangle. Consequently, the actual trajectory cannot proceed directly toward the center but instead exhibits a “zigzag” oscillation. Overshoot is highly likely to occur, so extremely high historical weights (β) must be assigned to shorten the step size and dampen oscillations, thereby achieving slow convergence.

2. Boundary Optimum

When the minimum lies outside the boundary and is subject to hard constraints, the direction of the true gradient component pointing outward from the boundary remains constant. This characteristic causes the FW algorithm to permanently lock the target corner $z_{k,1}$ at the boundary value (0.5) in the lateral dimension. The algorithm's trajectory exhibits distinct phases. Initially, before crossing the boundary, the trajectory exhibits a brief, smooth sprint; however, once the trajectory crosses the central axis, the reversal of the vertical gradient still triggers alternating tugs on the vertical corner points, causing the trajectory to take on a zigzag pattern. At this point, although the algorithm continues to oscillate, this oscillation occurs only along the boundary of the dimension $x_1 = 0.5$.